



AALBORG UNIVERSITET
STUDENTERRAPPORT

Technical Faculty of IT and Design
Aalborg University
<http://www.aau.dk>

Title: Sapheus – Stay Safe, Connect!

Theme: Connecting People

Project Period: February 2022 – May 2022

Project Group: IxD4-01

Participants: Eske Klintø Larsen

Jane Asp Strøm

Kevin Kularajah

Lucas Lindholm Reiding

Maja Hornbæk Kristensen

Sebastian Tor Douglas Stadsholt

Supervisor: Dimitrios Raptis

Report Page Numbers: 75

Data of Completion: 25 May 2022

Abstract:

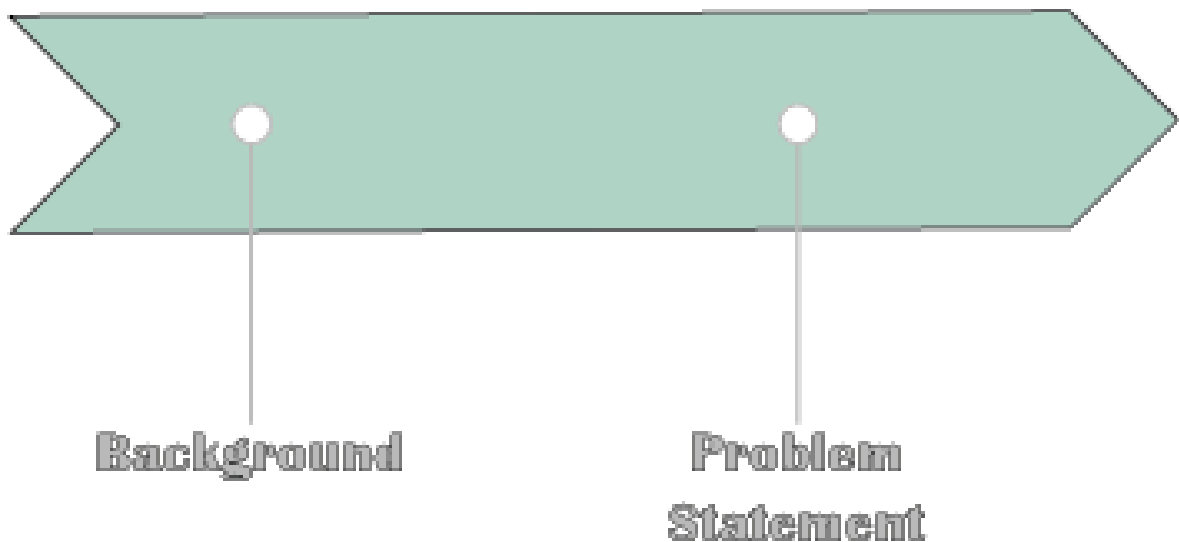
The purpose of this project is to design and code an interface in the theme of connecting people. When beginning this project, Mia Stevn disappears, heavily influencing our design process. We identify a problem regarding safety in nightlife and work from the problem statement: "How can we develop an interface that promotes safety and connectedness between friends?" The overall focus is to improve safety by connecting friends. Through our design process we create a prototype and afterwards we conduct an evaluation of the prototype. To design a prototype, we follow the Double Diamond of Design to guide our process in designing, researching the scope, involving users, coding and evaluating our concept. In this process, we involve users to improve our understanding of the problem field and to help evaluate our prototype. Our product is an app called Sapheus that helps friends know if everyone is safe. It fills a need expressed by our participants but needs some improvements before a potential launch.

Contents

1. Introduction	03
1.1 Motivation	04
1.2 Background	04
1.3 Problem Statement	05
1.4 Report Structure	06
2. Exploring the Scope	07
2.1 Approach	08
2.2 Research	09
2.3 Results from the Mind Map	11
3. Discover	14
3.1 Preliminary Questionnaire	15
3.2 Preliminary Questionnaire Results	19
3.3 Interview	20
3.4 Interview Results	21
4. Define	22
4.1 Device Choice	23
4.2 Focus	24
4.3 Concept	29
4.4 Adobe XD Test	32
4.5 Iterated Design	34
4.6 Naming the App	35
5. Develop	36
5.1 Applicable Theory	37
5.2 Code	38
6. Deliver	56
6.1 Questionnaire	57
6.2 Questionnaire Results	59
6.3 Usability Testing	62
6.4 Usability Test Results	63
7. Discussion	65
7.1 Scope	66
7.2 Nightlife Setting	66
7.3 Android Studio	66
7.4 Evaluation of Sapheus	66
8. Future Works	68
9. Conclusion	71
Bibliography	73

1

Introduction



The purpose of this project was to code a concept that fit within the theme of connecting people. This was done by exploring the potential scopes, collaborating with users, iterating through designs, and coding the chosen design derived from these iterations. In this chapter, we introduce the problem and relevant background information, that inspired this project. Next, we formulate a problem statement. Lastly, we outline the structure of this report.

1.1 Motivation

For this project, we were prompted to design and develop a functional, interactive prototype of an interface for a specific case within the theme of connecting people. From this theme, we discover design requirements that are implemented in the interface. In February 2022, when we were introduced to this project, two young adults disappeared in Aalborg during a night out with their friends, one entered a pirate taxi and one fell into the harbour. The specifics of these cases are detailed in the Background section below. This affected nightlife in Aalborg in its entirety, but it also affected the case within which our interface was developed. Upon the disappearance of two young adults in the same weekend, we agreed to use this semester project to aid in safety, especially during a night out, aimed at preventing more cases like these from being repeated. The specific cases that affected our decision are detailed in the following section.

1.2 Background

On February 3rd, 2022, after a night out in Jomfru Ane Gade, Aalborg, 21-year-old Oliver Lund disappeared. He was last seen late at night by the harbour, and the police worked from the conjecture that he had fallen into the harbour rather than being the victim of a crime. Two nights later, on February 5th, 22-year-old Mia Stevn also disappeared after a night out. With two young adults disappearing within 48 hours of one another, the police became heavily involved. After Mia's disappearance, the police quickly released that Mia's case differed from Oliver's because, it was clear to them, that Mia could be the victim of crime, because of her disappearance. She was last seen on surveillance tape from a parked car at 6 AM on February 6th. On the tape, Mia was seen, alone, getting into an unknown car, after which she disappeared. The car that Mia entered, was later discovered to be a pirate taxi. Mia was alone at the time she was last seen, since her friends had left earlier (Christensen, 2022a). In an attempt by the police to find Mia, the public was asked for help in finding her by saving their surveillance footage from the night, and any pictures or videos recorded in and around Jomfru Ane Gade (Clante et al., 2022). On February 9th, two men were arrested and later charged with murder while the police continued their search of Mia (Nordjyllands Politi, 2022). The day after these arrests, on February 10th, parts of a body were found in Dronninglund Forest, and it was confirmed to be Mia on February 12th (Christensen, 2022b). On April 14th, Oliver was found dead in Aalborg Harbour. He is suspected to have drowned (Jørgensen, 2022). At the time of this project, February 2022 through May 2022, the investigation surrounding Mia's death is still ongoing (Clante et al., 2022). The uncertainty around both disappearances prompted a conversation between the citizens in Aalborg and the city hall officials regarding safety, specifically in Aalborg's nightlife. City hall suggested how to improve the perceived safety in Aalborg (Jungersen, 2022). The police decided to be more present in and around Jomfru Ane Gade due the disappearances, and more initiatives regarding the physical safety present in Jomfru Ane Gade were either implemented or increased (Strømgaard, 2022). A great example of this is a group of volunteers called Natteravnene.

Their purpose is improving the security and well-being in nightlife. After these disappearances, they got many new volunteers in Aalborg. In addition to Natteravnene, a new volunteer-run project opened in Jomfru Ane Gade, called Safe House. Safe House aims to increase safety in nightlife by helping people call a taxi, charge their phones, or help with minor injuries (Julsgaard, 2022). With this understanding of a lack of safety present in nightlife, we proceeded to formulate our problem statement, which created the basis of all work conducted during our design process. The problem statement is explained in the following section.

1.3 Problem Statement

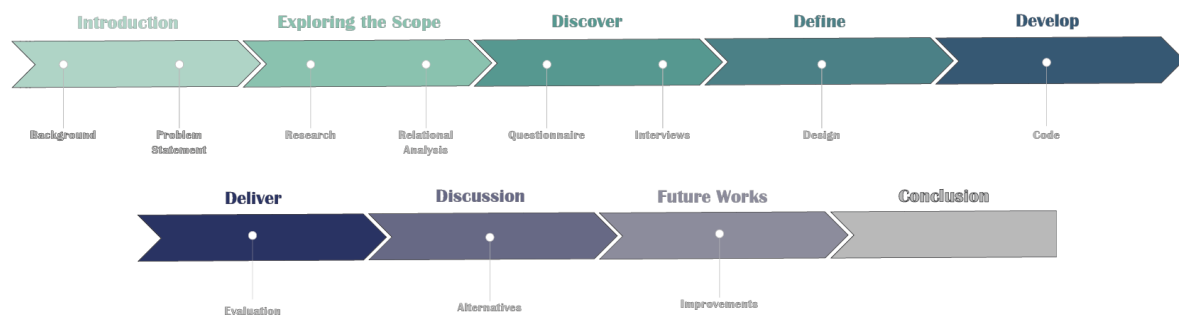
With the horrendous act of cruelty tied to the Mia case, we realized that we could create an interface that connects people by grouping friends together, and subsequently establishing a shared safety network that allows friends to help each other. With this basis established, we framed our design process and coding around the following problem statement:

How can we develop an interface that promotes safety and connectedness between friends?

With this problem statement established, we proceeded to define what friends and connectedness meant for our project. For the purposes of this project, we defined friends as people an individual would trust and depend on during a party. We defined connectedness as the feeling of being connected. It was important for us to establish these definitions, so that we could work from the same understanding of the underlying principles that our concept would rely on. As you will read in this report, our concept has been developed specifically for the nightlife setting, but we realize that it is applicable in multiple aspects of life, therefore the constraint of nightlife has been removed from the problem statement. With this clarified, we began developing our concept, aimed at establishing a safety net between connected friends, our process of which is detailed in this report. A brief overview of our process, which translates into our report structure, are detailed in the following section.

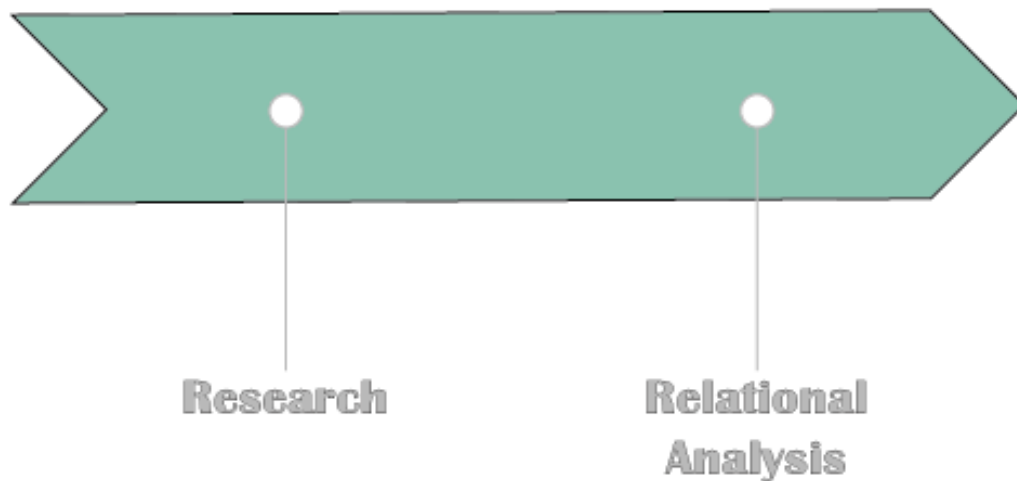
1.4 Report Structure

This report is a detailed description of our approach in designing and developing a concept that connects people and promotes safety in nightlife. This report is written in chronological order, directly depicting each step in our process with reference to the theory applicable for that specific approach. The applicable theory and our preliminary research to understand the problem scoped are detailed in chapter 2, Exploring the Scope. With a new understanding of the scope, we entered the Discover phase of the Double Diamond, where we sent out a questionnaire and conducted interviews. This was done to inquire about the general habits and safety related to a party. With these data collections, we were able to understand our problem area better, and the specific aspects of partying that cause the greatest concern. All of this is detailed in chapter 3, Discover. With data collected, we moved into the Define phase, where we used our interpreted data to sketch design ideas aimed at helping relieve some of greatest concerns associated with partying, detailed in chapter 4, Define. To move into the second diamond, we used our design from the first diamond, to create the foundation for the development of our concept and iterated through the design as needed. This process is detailed in chapter 5, Develop. To finish off our design process, we evaluated our design prototype, explained in chapter 6, Deliver. To discuss our process and other methods we could have applied, is detailed in chapter 7, Discussion. Future works that could improve the concept are detailed in chapter 8, Future Works. To round off this report and our process, the conclusions we have drawn from this process are detailed in chapter 9, Conclusion



2

Exploring the Scope



To start our process of understanding the scope of this project, we explored iterative design processes we could follow. Based on our knowledge of this project, we decided to follow the Double Diamond of Design, which is detailed in the first section of this chapter. Prior to entering the Double Diamond, we conducted exploratory research, to establish a shared understanding of the scope within which we design potential concepts. Rather than designing ideas from our own understanding and sharing them with one another within the group, we wanted to build a shared foundational knowledge on which we could build design concepts. Another advantage of this was allowing all our concepts to fit within the same scope, meaning that aspects from various concepts could be mixed, since all concepts had the same foundation. To create this shared foundation, we researched what it means to connect people and on what basis people connect. The theory related to Double Diamond of Design, our process and resulting knowledge are detailed in this chapter.

2.1 Approach

For this project, we have followed the Double Diamond of Design, established by the British Design Council in 2005. The Double Diamond of Design sections the design process into concrete stages, allowing designers to explore ideas, establish design criteria, develop a design based on prior ideas and criteria, and evaluate the design. This design process is separated into two diamonds, creating a distinction between phases of the design process that centre around divergent and convergent thinking. The Double Diamond of design can be seen below. (British Design Council, 2022a)

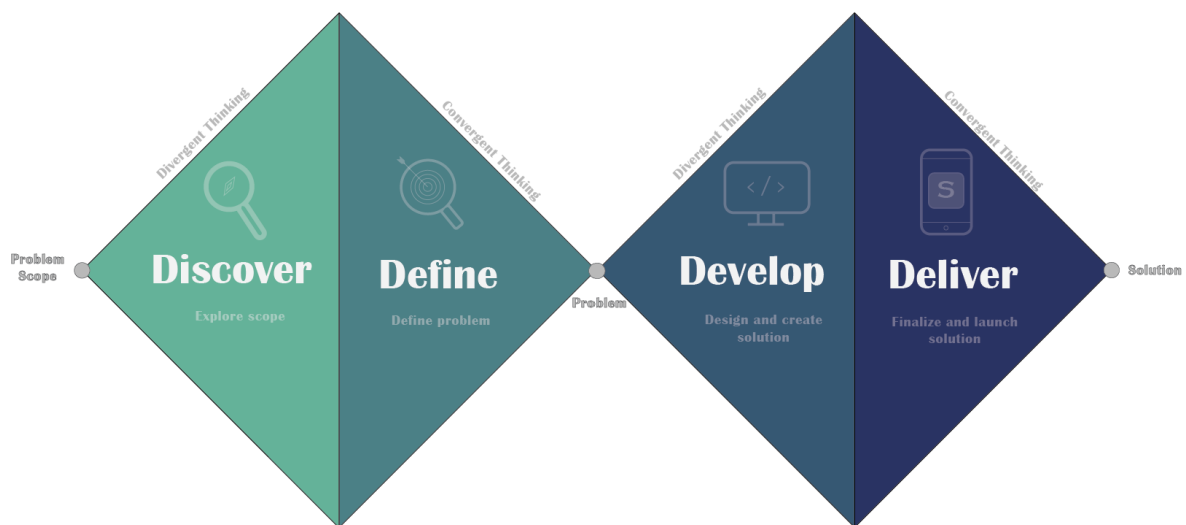


Figure 2.1: Double Diamond of Design according to the British Design Council.

The Double Diamond of Design has four distinct phases: Discover, Define, Develop and Deliver. These four phases are sectioned off into two diamonds, Discover and Define occupy the first diamond, while Develop and Deliver occupy the second. "The two diamonds represent a process of exploring an issue more widely or deeply (divergent thinking) and then taking focused action (convergent thinking)" (British Design Council, 2022a). According to the British Design Council, "the first diamond helps people understand, rather than simply assume, what the problem is" (British Design Council, 2022a). The first diamond is used to explore ideas and define the framework within which the design concepts should remain. The second diamond has the purpose of "[encouraging] people to give different answers to the clearly defined problem, seeking inspiration from elsewhere and co-designing with a range of different people" (British Design Council, 2022a). With reference to the ways of thinking mentioned above, the Discover and Develop phases employ divergent thinking, meaning that work is directed at exploring concepts and ideas. This can be seen in the figure above, since both phases start from a solid point and then the triangle opens, depicting this exploration of ideas. The Define and Deliver phases employ convergent thinking, since they both use the concepts derived from the previous stage and use it to solve the problem that the concept was designed to solve. (Shrestha, 2017) The first diamond has two phases, Discover and

Define. According to The British Design Council, the Discover phase is used to explore a topic or issue. People who are affected by this issue are involved in this phase of the double diamond to ensure that the product developed is clearly focused on the needs of users, relating back to developing a product that is based on actual necessity and not the designer's assumptions. (British Design Council, 2022b, pp. 8-13) The Define phase is used "as a filter where the review, selection and discarding of ideas takes place. This is where findings from the Discover phase are analysed, defined and refined as problems, and ideas for solutions are pitched and prototyped" (British Design Council, 2022b, p. 14). With the completion of the first diamond, the second diamond is entered through the Develop phase, where the purpose is "to prototype and iterate the concept to get it as close to an end product or service as possible" (British Design Council, 2022b, p. 21). Once the concept is developed, the last phase of the Double Diamond, the Deliver phase, is entered. In the Deliver phase, "the final concept is taken through final testing, signed-off, produced and launched" (British Design Council, 2022b, p. 23). By following the Double Diamond of Design, we were able to get structure in our process of working with our concept. Having defined the design process we follow in this project, we initiated research aimed at understanding the scope of this project.

2.2 Research

To start our process in working with this project, we brainstormed how people connect. This was done to ensure that our concept would fit within the overall theme of connecting people and to establish a shared understanding regarding what kind of connection we wanted our concept to facilitate. In doing this, we realized that the possibilities of connecting people seemed endless, which prompted us to define what it means for people to connect, rather than simply brainstorming ways to connect people. To understand how people connect, we researched and discussed how people connect and made a mind map, grouping together research and ideas based on similarity, which can be seen in figure 2.2. Four categories regarding how people connect emerged from our mind map: how people connect, where they connect, on what basis they connect, and why connections are important; represented by the blue squares in the mind map. The yellow squares contain the actual research we had conducted or ideas we had noted.

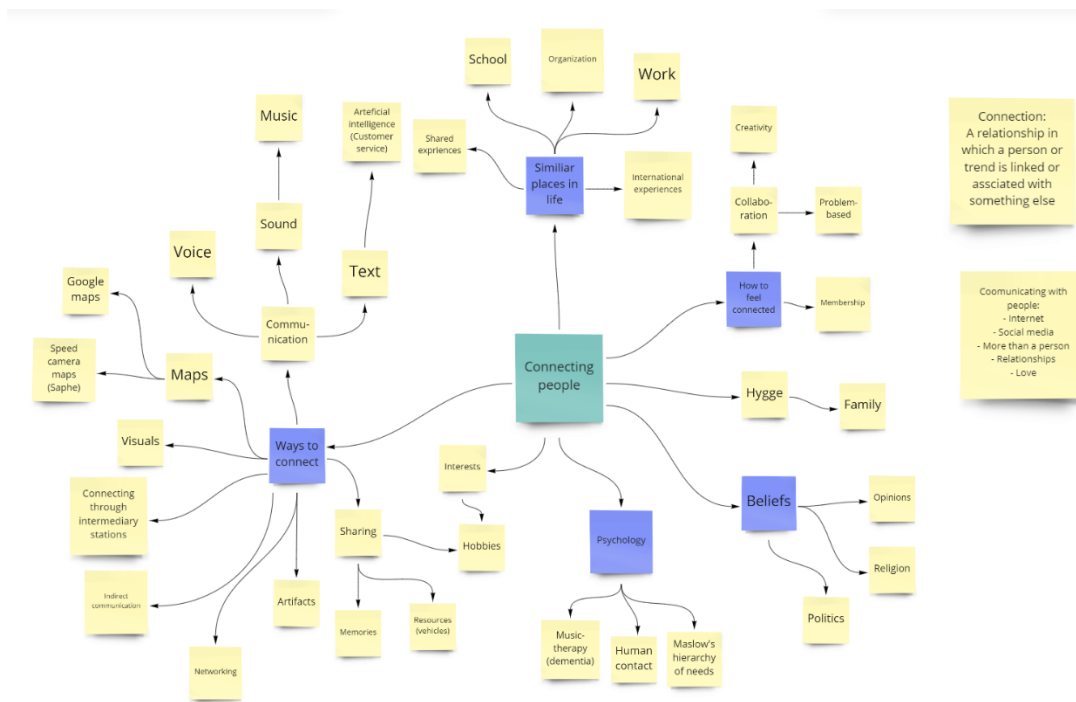


Figure 2.2: Mind map of preliminary research focused on connecting people

To further assess whether the categories we had defined fit within the scope of *connecting people*, we conducted a content analysis. Content analysis is “used to determine the presence of certain words, themes, or concepts within some given qualitative data” (Health, 2022). There are two forms of content analysis, conceptual analysis and relational analysis. Conceptual analysis regards the frequency of a concept within text. Relational analysis examines the relationship between concepts, where “[individual] concepts are viewed as having no inherent meaning and rather the meaning is a product of the relationships among concepts” (Health, 2022), creating codes for the established relationships. Based on these definitions, we conducted a relational analysis because, with the incorporation of codes, it helped us find themes within our previous mind map, allowing us to create connections between codes that encompass multiple concepts found within our research. We coded our mind map because it allows us to refer to it once we need to assess whether our design concepts fit within the theme of connecting people. This was also done to enhance our understanding of the overall problem. To start our relational analysis, we defined our research question: how do people connect? This research question needed to be established to ascertain whether each theme and research concept was relevant within the scope of this project. Step 2 in a relational analysis is to identify codes and categories of the text, making it easier to relate certain codes. We had already done this during our research and ideation brainstorm, allowing us to transfer these previously established codes to the relational analysis. After coding our ideas and our research, we proceeded to step 3, exploring the relationships, to identify three overall themes and categorize all of our ideas and research, which can be seen in 3 blue squares in the relational analysis mind map below. Once these three overall categories were established, our ideas and research were placed within the appropriate

category, we identified that certain notes, the yellow squares, could be further categorized, as seen in the green squares. This follows the 4th step of a relational analysis, coding the relationships. This further categorization depicts how people connect in all aspects of life, further showing the vast nature of connecting people. The presence of the blue squares depicted how vague aspects of connecting people are, the green squares were introduced to further categorize the mind map. By having these green subcategories, we were able to move onto step 5, where we explored the relationships identified, progressing us into step 6, analyzing collected data, allowing us to decide what categories of connecting people could be applied to our concept. The mind map resulting from the relational analysis can be seen below in Figure 2.3. (Health, 2022)

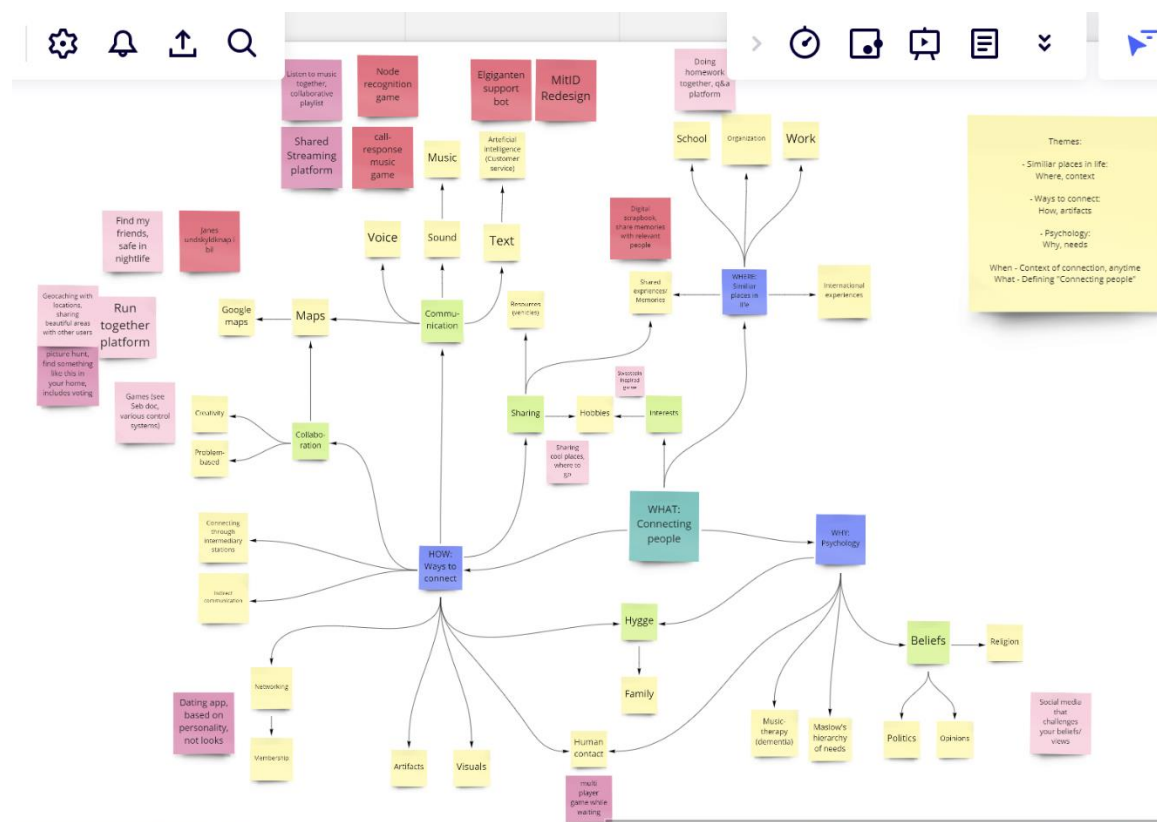


Figure 2.3: Mind map organized after thematic analysis

2.3 Results from Mind Map

As previously mentioned, three overall themes, the blue squares, resulted from the relational analysis. The resulting themes were: where people connect, how people connect, and why people connect. Based on our research and our own ideas and experiences of connecting with others, we were able to categorize all our notes into one of these three themes, which is detailed below.

- Where people connect: Based on the notes that are related to this theme, we discovered that people being in similar places in life was a common foundation on which people connected. Things like shared experiences and common daily

activities connected people, as are detailed in the yellow squares connected to the Where blue square.

- How people connect: This theme concerns notes that detail ways people connect. Topics like communication, collaboration, sharing and “hygge” are some ways in which people connect. This theme is used as inspiration for ways we can include connecting people into our product.
- Why people connect: People connect on a basis of shared beliefs, since a predetermined foundation for the connection has been established. According to Abraham Maslow, the creator of Maslow’s hierarchy of needs, humans have 5 levels of need, sectioned into lower-level needs and upper-level needs. The 5 levels of need, listed from lower to upper needs are: physiological, safety, love and belonging, esteem, and self-actualization. The first three levels are considered part of lower-level needs, and the remaining two are upper-level needs. Once the lowest level of needs, physiological needs, are met, then an individual can attain the needs of the level directly above. So, when the physiological needs, health, rest, nutrition, shelter and clothing, are met, an individual can proceed to fulfill safety needs, protection and emotional welfare. Once an individual is safe, then they can fulfill love and belonging needs. This is where genuine human connection is attained, and connecting people is placed. With the lower-level needs fulfilled, an individual can proceed to attain the upper-level needs. Since connecting with people is a lower-level needs, humans require connections with others to be able to attain the needs of esteem and self-actualization. This aids in illustrating the importance of connecting people. Connections are a lower-level needs, meaning that humans can’t flourish if their need for connecting with people isn’t met, illustrating the importance of connecting people. (MasterClass Staff, 2020)

After completing the relational analysis, and having these themes and categories established, we began designing concepts that could fit within one or more of the themes detailed above. These ideas were then added to the brainstorm to see how the ideas would map out compared to our relational analysis. All ideas present on the mind map above were judged based on their relevance within the themes related to connecting people. If an idea didn’t fit within any of the themes, it was discarded since it didn’t respect our defined aspects of connecting people, a requirement for the designs. If an idea was within the scope of the project, it was placed on the mind map based on the theme it fit within, depicted through the various shades of red squares. The exclusion process mentioned above was the sole criteria we used to decide which ideas were included on the mind map. Since we hadn’t decided on a concept yet, we allowed our own opinions to influence the topic we would work with. Since this report reflects a semester project, we found it important that we were passionate about the topic we were working with. Upon discussion regarding what we wanted to work with and what concepts we could

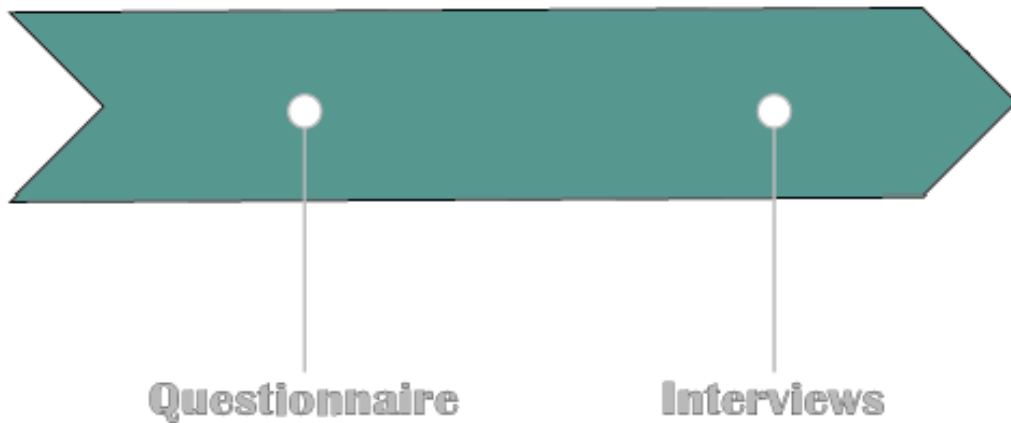
develop with our knowledge of programming, we were able to further distinguish the red squares:

- light pink squares: ideas we liked
- purple squares: potential features
- red squares: ideas we didn't like

After categorizing potentials concepts, we initiated a discussion about what concept we wanted to further develop. With the surfacing of the Mia case, we discussed the advantages and disadvantages of developing the Safety in Nightlife concept, since it was directly derived from Mia's disappearance. With this discussion of advantages and disadvantages, we decided to further develop this concept because we were all passionate about making something that could encourage safety, while connecting people, hoping our concept could help others in situations like Mia's. With our scope defined, we started collecting data regarding habits when partying from fellow students in IxD4, to inquire about their behaviours when partying, to assess if this concept could be of help, ideally minimizing cases like Mia's in the future. Our data collection process is detailed in the following chapter, where we enter the Discover phase.

3

Discover



To enter the Double Diamond of Design, we started by discovering the requirements we had to respect, and the general behaviour and safety associated with partying. The main requirements we had to work within were: the theme of connecting people and our product had to be developed in a coded environment. In the Discover phase, we involved participants to learn more about the habits of people who party and to identify whether our defined problem was a concern that the participants had. To inquire about this, we sent out a questionnaire to gather data regarding the overall habits when partying, and then we conducted some interviews to further inquire about specific habits and worries related to partying mentioned in the questionnaire. Our process regarding this data collection, alongside the results derived from these, are detailed in this chapter.

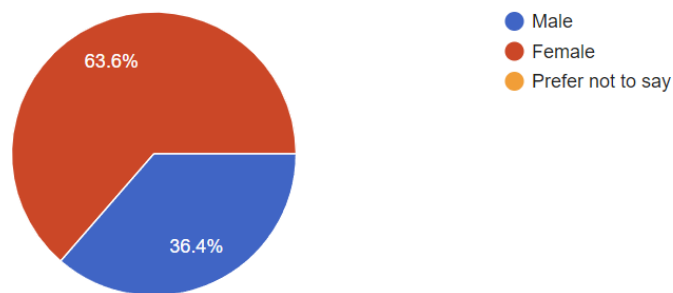
3.1 Preliminary Questionnaire

To start collecting data, we developed a questionnaire that inquired about partying habits of our fellow students in IxD4. By sending the questionnaire to our fellow students, estimated to be around Mia's age at the time of her death, we sought to use this demographic as a sampling indicative of the target group of our concept. The purpose of the questions and the responses collected are outlined below, and the results are interpreted in the following section below. In the following section, the target group is defined as well. To start the questionnaire, we inquired about the participant's gender and age, see figure 3.1. These questions were included to aid in the definition of our target group. The responses from these questions are depicted below.

Gender?

11 responses

 Copy



Age?

11 responses

 Copy

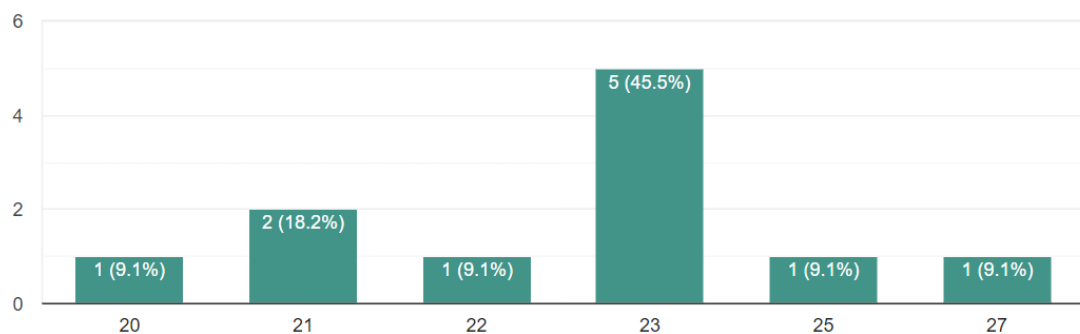


Figure 3.1: Gender and Age responses from questionnaire.

With gender and age established, we proceeded with questions regarding the participant's habits before, during and after a party. Additionally, we inquired about modes of transportation and general methods of sharing information. To learn about general behaviour before a party, we constructed three questions to understand their approach to partying, see figures 3.2, 3.3, and 3.4. The participants were asked what they do before a party, how they get to the party, and if anyone knows they are going to a party. All participants had the option to select more than one answer, since we acknowledge that not every party is the same, thereby altering their behaviour to match certain types of parties. Below, the responses to these three questions can be seen.

What do you do BEFORE a party?

 Copy

11 responses

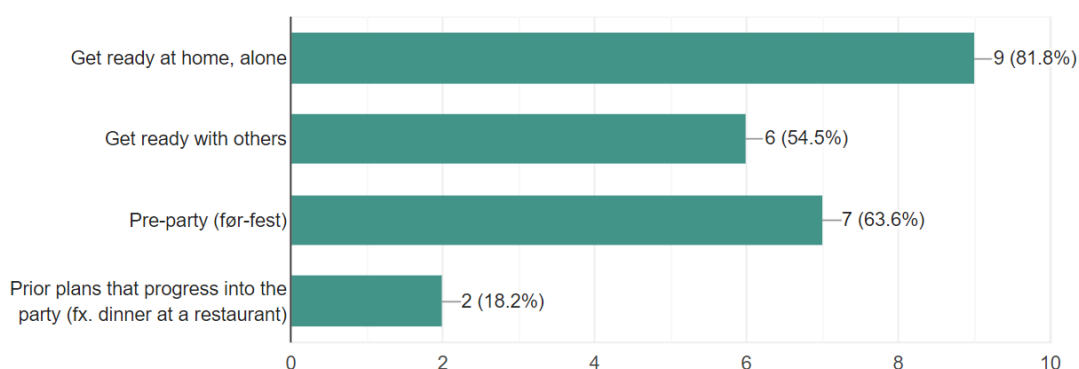


Figure 3.2: responses to question: What do you do before a party?

Mode of transport to a party

 Copy

11 responses

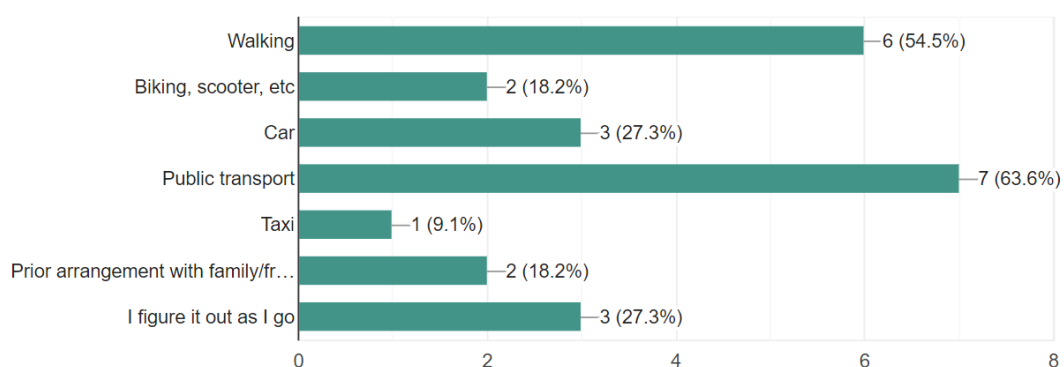


Figure 3.3: responses to question: Mode of transport to a party?

Does anyone know you're going to a party?

 Copy

11 responses

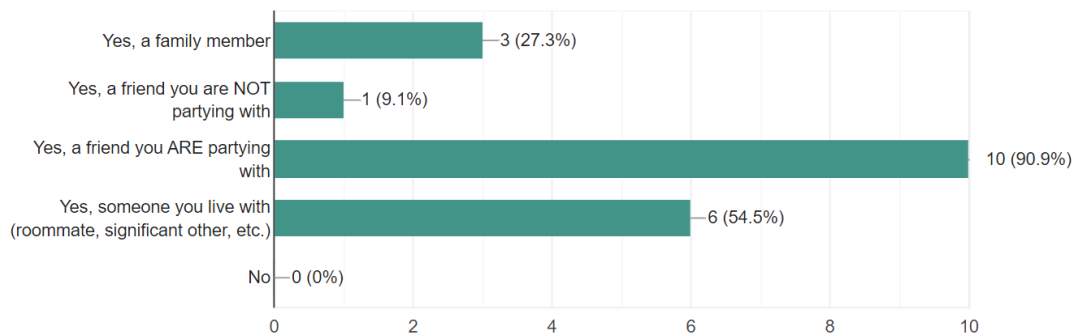


Figure 3.4: responses to question: Does anyone know you're going to a party?

With questions about behaviour prior to a party established, we inquired about the participants behaviour during a party, see figure 3.5, with the question of: what do you do during a party? This question was included to see if most responses showed a tendency towards partying with people they know, partying with people they do not know or remaining independent. This question was aimed at showing if there was a sense of connectedness when partying, which we could use in the development of our concept, if this sense was present. Below, the general responses to this question can be seen.

What do you do DURING a party?

 Copy

11 responses

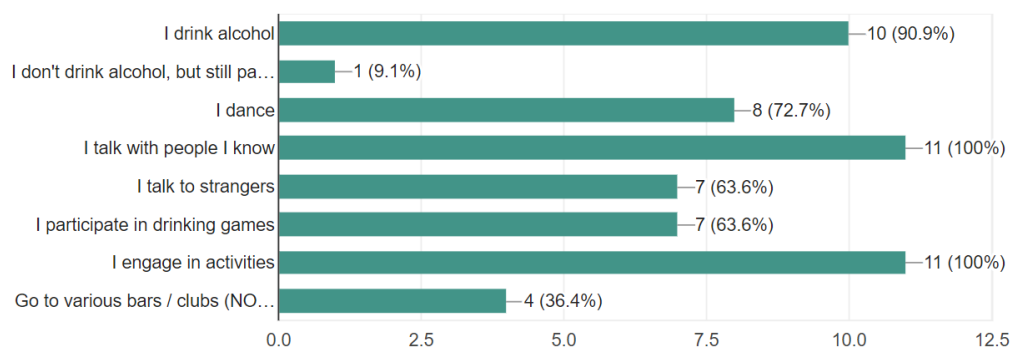


Figure 3.5: responses to question: What do you do during a party?

To finish the questionnaire, we developed questions to inquire about the participants behaviour after a party, specifically when returning home, see figures 3.6, 3.7, and 3.8. It was important to us to include questions that detailed behaviour after a party, because this was when Mia entered a pirate taxi and disappeared. To inquire about behaviours after a party, we asked questions about how the participants left the party, how they got home and if anyone knows they've gotten home. This last question was included because this was how the Mia case was started, none of her friends or family had heard from her

after the party and this caused concern and was later reported to the police. The responses to these questions can be seen below.

How do you leave the party?

[Copy](#)

11 responses

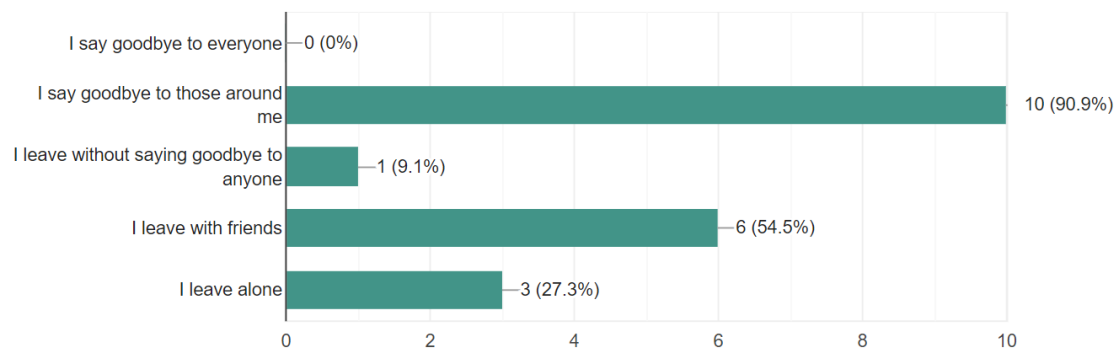


Figure 3.6: responses to question: How do you leave the party?

Mode of transport home (regardless of where you are staying)?

[Copy](#)

11 responses

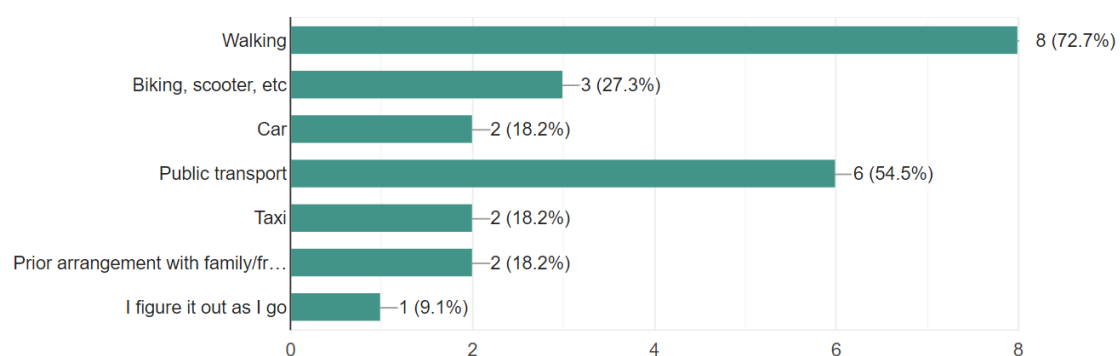


Figure 3.7: responses to question: Mode of transport home?

Does anyone know you've gotten home?

[Copy](#)

11 responses

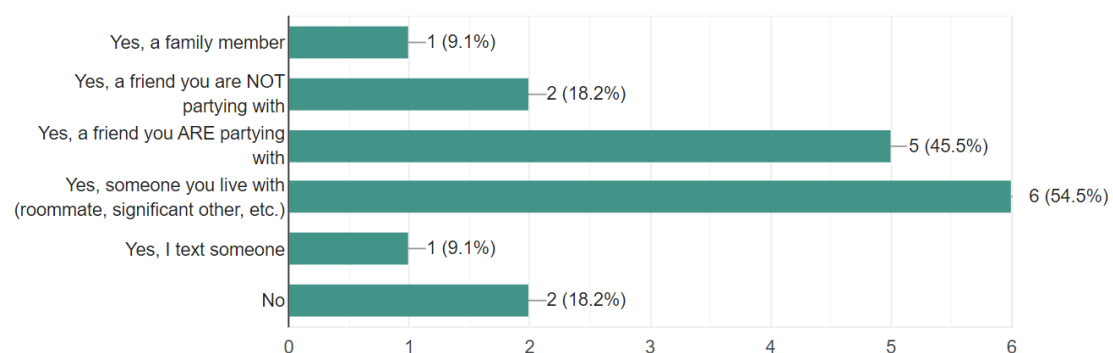


Figure 3.8: responses to question: Does anyone know you've gotten home?

With the questionnaire constructed and the data collected, our interpretation of the data is detailed in the following section.

3.2 Preliminary Questionnaire Results

Based on the questionnaire, we saw that both genders responded, all fitting within the age range of 20-27 years old. With this age range, we defined our target group as anyone between the ages of 18-30, since we wanted to add a bit of leverage between the minimum and maximum participant ages. We decided to add 3 years on either end, meaning that our target group would be between 17-30, but the legal age for purchasing alcohol at a bar or club in Denmark is 18 (Møller, 2016), so we set that as our lower limit. With regards to behaviour before a party, we learned that most people get ready for a party with others, which could be used as the time frame when digital connections between friends was established. Every participant said that they always tell someone they are going to a party, a fact we can use to establish a safety net between the person partying and their relatives or friends. To get to the party, most people walk or take public transport, knowledge we couldn't utilize as is, which is why we use it as the foundation for part of the interviews, which is detailed in the following section. As previously mentioned, a question was included in the questionnaire regarding the participant's behaviour during a party, specifically regarding who, if anyone, the participant spends time with at a party. Here, we learned that most of our participants spends time with others at parties, seldom choosing to party alone. This information is important to us, since it shows us that we can create a digital connection between people to aid in safety, since we now know that most people party with others. With regards to the participant's behaviour after a party, we learned that most of our participants notifies either one person or a group of people that they are leaving. We found this interesting and transferred this knowledge to the interviews to further inquire about the importance of this and why people do this. We learned that most people either walk or take public transport to get home from a party, the same as how most people get to a party. This wasn't of specific interest, but we transferred this knowledge in the interviews, to further learn how people get home and if safety, in any aspect, is of importance. Lastly, we asked if the participants notify anyone that they've gotten home, and some people said they don't, which caused curiosity. With this curiosity, we also transferred this to the interviews, to assess if it is of importance to our target group to know if others have gotten home and if it is of importance to them to let others know that they are safe. With this knowledge, we developed an interview, to create a deeper understanding of the behaviour and safety related to partying, with specific emphasis on the behaviours related to leaving a party and going home. Our interviews and the results are detailed in the following sections.

3.3 Interviews

With the responses from the questionnaire, we know what people tend to do before, during and after a party. This created the foundation for interviews held with 4 participants in the target group. With the interviews, we aimed to get a better understanding of the problem by understanding why people exhibits these behaviours. From the questionnaire, we learned that most people walk or use public transportation to get to a party. We asked the interviewees if they were part of the group using these methods to get to and from a party and why this was their preferred method. The interviewees said that they walk, if the distance is manageable on foot, and use public transportation or have their partner drive, if the party is farther away. When asked why these are the popular choices, the interviewees expressed a concern for the prices related to other modes of transportation and therefore use the cheapest possible options, walking for short distances and buses for longer distances. With specific regard to getting home from a party, the interviewees noted that they were more likely to walk home than to a party because this was easy and cheap, since the bus is not always the fastest mode of transport at night. If possible, they arrange to walk home with someone from the party who lives in the same area. One interviewee said that they ensure to text their partner when they leave the party and begin walking home. This was described as a safety precaution, since their partner would know that something was wrong, if the walk home took substantially longer than it was supposed to. The interviewees did not feel the need to tell anyone that they had gotten home safe, except the ones they live with, if any. One interviewee explained that it was customary for them to text someone from the party when waking up the next day to thank them for at great party, subsequently telling that person that they've gotten home safely. They said that if someone didn't respond to this text within the day, concern would arise. Two interviewees noted that it was more important for them to know if their friends got home safely, rather than notifying their friends of their own safety. This was mostly important to them if the friend lived alone, because no one would know if they got home. This usually results in the participants asking their friend, when leaving, to text them once he/ she has gotten home. If the friend forgets, they don't get concerned right away but inquires about their well-being the day after. Even though they do not expect a text from the person who got home, they say that it is nice to know when their friends get home. Lastly, we wanted to hear the interviewees worst story about something that happened in relation to a party, to get a better understanding of certain problem areas related to partying. This could be their own story or something that happened to someone they know. The stories were not similar. One had lost both their phone and their keys and had to go to a friend's home. Another participant had a friend who got very sick because of a spiked drink and arrangements of transportation home had to be made quickly. The last stories detailed friends getting lost and waking up on the street. This illustrated a sense of danger related to partying, leading us to initiate a discussion as a group, specifically about the interview responses and how we could use these responses as a framework

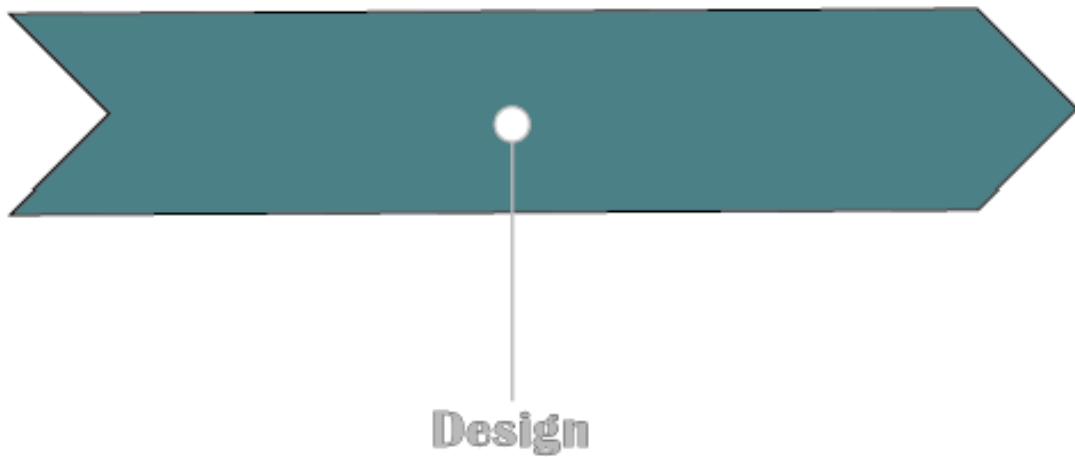
for our concept. The results from this discussion, derived from the interviews, are detailed in the following section.

3.4 Interview Results

With the questionnaire sent and the interviews conducted, we learned about certain habits and preferences related to partying. We discussed what aspects of the collected data we could use to develop our concept, and what areas of a party that this concept needs to help. With the data, we learned that most safety issues occur after a party, or towards the end of a party. We learned that most people are more concerned with the safety of their friends, rather than their own. Since this fact was mentioned by two interviewees, we concluded that, if there is a friend group, as the basis of the connection, implemented in our concept, we could aid in the shared feeling of safety, since everyone knows their friends are safe. Additionally, we learned that leaving the party and going home has certain risks attached, therefore, we wanted our concept to establish a safety net within a friend group, specifically targeting the transportation time when going home. With these facts established, we moved into the Define phase, where we sketched and iterated through multiple concept designs, aimed at respecting the two take-aways from the questionnaire and interviews. The process of designing concepts and further developing design requirements are detailed in the following chapter, Define.

4

Define



Having established a problem, we wanted our concept to help solve, and assessed the validity of our concept within its intended usage, we moved on to defining the parameters of the project, moving us into the Define phase. In our Define phase, we use our findings from the Discover phase to define and refine design requirements ideas. These ideas are reviewed, selected, and discarded as needed, until we have a design that fulfills various requirements, that is brought into the second diamond, and developed, see chapter 5 (British Design Council, 2022b, p. 14). First, we discuss various interaction devices that our concept would be designed and coded for and choose the format in which our concept will be developed. Then, sketches with various focuses are presented, establishing the foundation for which concept requirements are developed upon. Then, additional sketches are discussed with regards to their fulfillment of the previously presented concept requirements, and, from these sketches, a design is chosen. Next, this design is tested through an Adobe XD prototype, aimed at testing the various functionalities included to accommodate to the concept requirements. Then, an Adobe Illustrator model of the interface design is presented, and the chapter concludes with an account of how our concept was named.

4.1 Device Choice

Prior to designing any concept, we initiated a discussion aimed at deciding what device our concept would be coded for. Listed below are the options discussed and underneath the list are the arguments for each device.

- An app
- A communal screen, proximity
- A wristband, always available when partying
- A smartwatch feature, easily accessible.

To outline our discussion, the advantages of making an app are recognizability and prior experience with other apps makes the usage of the app easier to navigate, regardless of inebriation. This aligns with Jakob Nielsen's "Consistency" usability heuristic, since the user knows that experience will positively affect their usage with our new app (Nielsen, 1993, p. 20). The main disadvantage of making an app is the requirement of installing the app prior to a party, which requires planning. Additionally, if the user loses their phone, the app becomes useless. The second point of discussion was a communal screen. The main advantages of a communal screen are proximity and independence, specifically from the user. As the possibility of losing one's phone was a disadvantage of an app, this possibility is eliminated if the interface is made for a communal screen. The main disadvantage of a communal screen is its lack of movement since the purpose of the interface is to increase safety regardless of the user's location. An unsafe user might not be able to get to the screen, meaning that the interface won't have any practical usage regarding safety, unless the user should experience danger within reach of the screen. The third device that was discussed was a wristband. The advantage of a wristband is its availability when partying. A wristband is secured around the user's wrist, making it significantly harder to lose than, for example, a phone. The biggest disadvantage of a wristband is the requirement of universal implementation, meaning that most people at a party needs to wear a compatible wristband while being sober enough to remember what each response from the wristband means. And the last device discussed was an application made specifically for smartwatches. The main advantage of coding an application for a smartwatch was accessibility, since it is fastened on the user's wrist, like with the wristband. But, as an added advantage of using a smartwatch, a smartwatch has a screen on which notifications can be displayed, making it easier for the interface to interact with the user without creating excessive inconvenience. A large disadvantage of coding a smartwatch application, was the decreased user market compared to making an app or a universal wristband. If we establish the concept for smartwatches, the usage of the interface is restricted to people with a smartwatch, which is a smaller market than that of an app for a smartphone (Feldmar, 2020). Based on these arguments, we decided to further develop this concept as an app due the familiar nature of apps and the app could reach a larger audience, subsequently leading to the conclusion that most people bring their phone with them while partying, meaning that the app will be more usable to a

larger audience than that of smartwatches and communal screens. Additionally, there is a certain familiarity with apps, leading to more people being able to utilize this interface, even when drunk. A communal screen does not facilitate safety while traveling home, which has become a central element of this concept, derived from the questionnaires and interviews, in which we learned that most people encounter problems during this time. Wristbands would produce unnecessary waste and there is a certain learning curve that drunk users would need to partake in, which seems unlikely. Less people have smartwatches making the consumer group smaller. A smartwatch also has a smaller screen, which could present a problem to drunk users. After deciding how our concept would be launched to our users, we intended to start sketching concept ideas. Upon further discussion regarding what should be sketched, we specified the focus of the app, since the functionalities of the app seemed random, if we didn't specify a focus. We proceeded to sketch different designs that focused on either partying or safety, our process and related discussions are detailed in the following section.

4.2 Focus

This section elaborates on our process in deciding the focus of our app. When initially designing this app, we quickly realized that making an app that focuses on both, makes the intended usage unclear, thereby making our design process confusing. To avoid this confusion for us and potential users, we initiated discussions and sketching rounds aimed at clarifying the focus of our app, partying or safety. First, our sketches and discussions related to a party-focused app are detailed, followed by the sketches and arguments for a safety-focused app. To conclude this section, we detail the outcome of the discussions and progress into sketches for the final design.

4.2.1 Party

Before discussing the requirements and sketches for a party-focused app, the purpose of a party-focused app was defined. The purpose of making a party focused app is encouraging digital and physical communication between friends, by bringing it to the forefront and enabling friends to interact with one another within the app. If our final concept centres around partying, build-in safety elements would have to run as background features that users would only uncover when needed. With specific reference to the physical design of the interface, the party-focused app should afford the happiness associated with being around friends. To allow for creativity within the sketching process, no concrete requirements were established, but, when sketching, the designs had to maintain a focus on social activities, like chatting with friends, finding one another and a game feature, intended as inspiration for fun activities that can be enjoyed between friends. The resulting main pages can be seen in figures 4.1 and 4.2.



Figure 4.1: Main Menu sketch 1

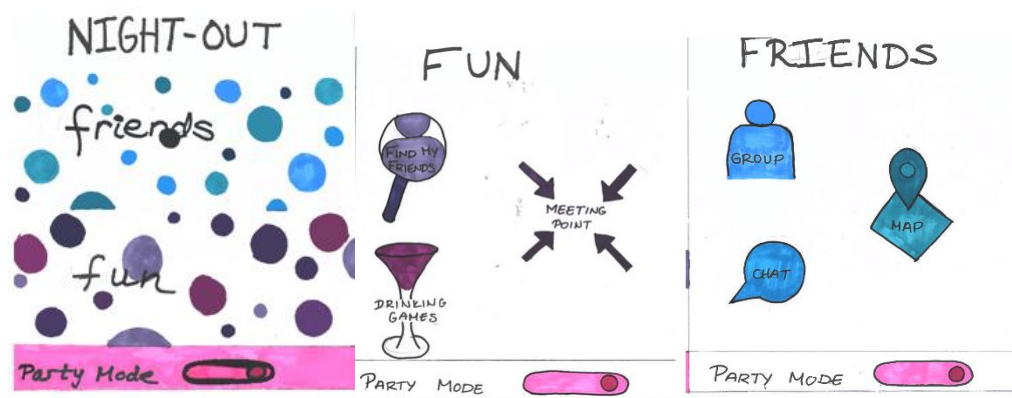


Figure 4.2: Main Menu sketch 2, with nested functionality pages

Figures 4.1 and 4.2 proposes two designs for a party-focused app. The main menu, seen in figure 4.2, has two buttons which are *friends* and *fun*, which lead to the pages with the same names. These pages contain all the functionalities included in the app, but each functionality is nested within one of two categories available on the front page. The main menu seen in figure 4.1 has all six functionalities available on the main menu, for easier access instead of nesting functionalities within subsequent pages, for example, the *friends* and *fun* pages. The functionalities made available in these sketches are detailed below. The main concept that these sketches are based on is a group-based connection, that aims at creating a safety net between friends while emphasizing the importance of connections when partying. The *fun* page, figure 4.2, provides several ways for users to engage in activities with each other. The same functionalities are directly available on the main page seen in figure 4.1, specifically in the right-hand column. With regards to the *fun* page, the user has three choices, which are *find my friends*, *meeting point*, and *drinking games*.



Figure 4.3: Meeting point



Figure 4.4: Find my friends

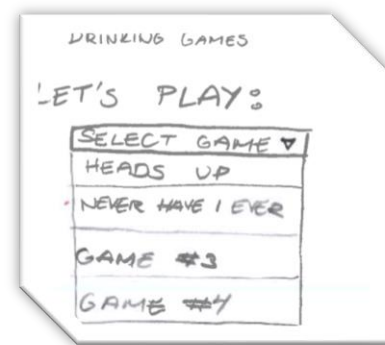


Figure 4.5: Drinking game

The buttons lead to the pages seen in figures 4.3, 4.4 and 4.5. The *meeting point* page, figure 4.3, makes the user set a location and time for a meeting point and the send button notifies their friends of the creation of the meeting point. This function was included to help promote a sense of connectedness within a friend group since we learned, from our interviews, that private parties are preferred because the interviewees find it frustrating getting lost from their friends when partying in public places. An alternative design for this page can be seen in figure 4.6 below. The *find my friends* page, figure 4.4, is used for displaying all the current friends in the group, their last location and a *where?* button, that redirects the user to a map, where a route to the friend is made. This function is included to make it easier for the user to navigate to their friends without having to find them on the map first. The *drinking games* page, figure 4.5, gives the user the ability to select a drinking game from a list of games that the app is intended to offer. This is included to give the user easy access to drinking games that can be played with their group. This was also included to provide additional incentive to download the app, since it has more diverse usages, ranging from finding one another to drinking games. With the party-focused sketches explained and their accompanying functions, the safety-focused sketches are explained in the following section.

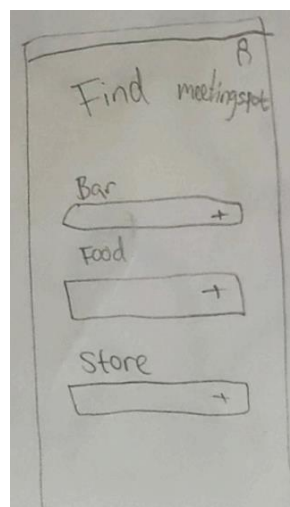


Figure 4.6: Meeting point page alternative

4.2.2 Party

To differentiate between the party-focused app and the safety-focused app, the focus of the safety-focused sketches had to be aiding in the group's collective safety. As in the party-focused app sketches, the establishment of groups, and creating a safety net within these groups, is a central concept in the functionality of all the sketches that are detailed below. With the party-focused sketches, it encouraged the user to involve their smartphone more when partying, but for the safety-focused app, we wanted to make an app that works silently during a party, where the user only needs to open it if safety becomes a concern. It should be an app that people download and then limited interaction is required to get the intended functionality. We were inspired by the COVID-19 infection tracking app "Smittestop", which links two phones that have been within two meters of one another for longer than 15 minutes, and notifies users when they have been near a person who has tested positive COVID-19 (Digitaliseringsstyrelsen et al., 2020). For the purposes of our app, the users need to create a group and have their friends join the intended group before going out to party. To start sketching safety-focused interfaces, we established some features that should be included in the sketches. The requirements were:

- *I'm home* button, to tell others you have gotten home
- Ability to read the status of friends
- SOS functionality, to request help should it become a necessity

The first two requirements were directly derived from our interviews, where we learned that our interviewees are more concerned with the safety of their friends, so we wanted our interface sketches to enable a safety net where every group member can see if other group members have marked themselves as safe, rather than remembering to send a text. The last requirement was included because we wanted a way to request help from one's friends, should that ever become necessary. With these requirements, we began sketching interfaces that focused on promoting a feeling of safety when partying.

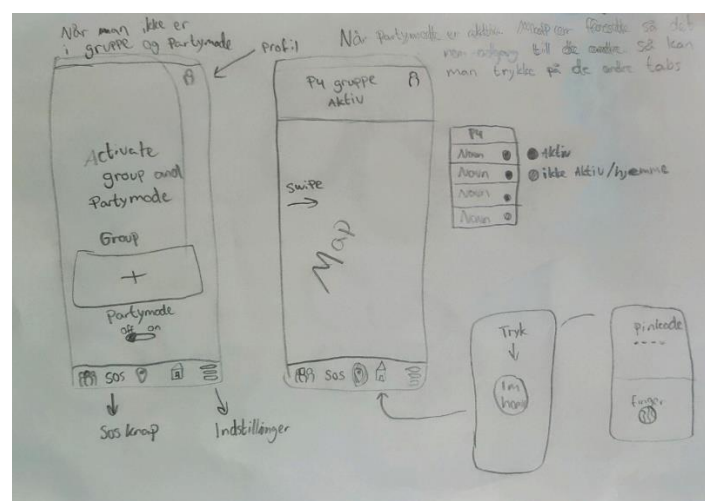


Figure 4.7: Safety-focused sketch



Figure 4.8: Alternative safety-focused sketch



Figure 4.9: Safety page

The first safety-focused sketch seen in figure 4.7, shows a design for the main menu and the map page. When the app opens, the user is welcomed with an *Activate group and party mode* request. The user must join a group and activate party mode to use the app, which is seen on the left side of the figure. At the top right corner of the pages there are a *profile* icon. At the bottom of the pages, there is a navigation menu that shows icons for the pages *Groups*, *SOS*, *Map*, *I'm safe*, and *Settings*. This makes different functions of the app more accessible. When a group is active, the user is met by the map, see the middle of the figure, where the user can swipe from the left to see the other group members and their status. This is to give them a shortcut to see their status. At the top, the active group will be shown. When the user presses the *I'm safe* button, the app wants to verify this activation with a password or fingerprint, see the right side of the figure. This is to make sure that the user doesn't press the *I'm safe* button by mistake. The button marks the user as safe and turns off their GPS location, no longer sharing it with the group.

Figure 4.8 introduces a design for the main menu. The main menu has two menu items, *safety* and *friends*. The *safety* button leads to the safety page, see figure 4.9. On the safety page, the user can press the *I'm home* button to mark themselves as safe. A *Help* drop-down box with several pre-made choices is also available on this page for the user to select if they feel unsafe and in danger. Various choices are given to the user as a shortcut to communicate their situation to their group and emergency contacts. The *friends* page follows the same design from the party-focused app, specifically in figure 4.2. With all the sketches completed, we discussed what focus was more relevant with regards to our interview data. Based on our interviews, we decided to further develop an app that was safety-focused. This was decided based on the interviews mentioning that

it can be difficult to read and notice messages, especially when under the influence of alcohol. We proceeded to define the requirements for our safety-focused app, which is detailed in the following section.

4.3 Concept

With the focus established, we proceeded to establish requirements for our app, based in our collected data and the previously conducted research, so another iteration of sketches could be entered. Before establishing any requirements, we returned to the questionnaire and interviews, as well as the study regulation in place for this project. Based on our data and the study regulation, three requirements emerged:

1. It must connect people
2. It must promote a sense of connectedness while partying
3. It must facilitate safety while people are returning home

The first requirement was established to ensure that all sketches respect the theme of connecting people, simultaneously, making users feel connected to one another and safe, translating into the second requirement. The third requirement was established based on our interview data. As previously mentioned, multiple interviewees detailed various agreements with partners and friends, aimed at ensuring the interviewee's safety when returning home from a party. To aid users in feeling safe, our app designs had to facilitate this, rather than requiring users to have their own safety precautions established outside the scope of the app. With these three requirements established, we sketched ideas that would fit within these requirements. The resulting sketches were made during a group discussion regarding what features respected these requirements, as detailed in the following section.

4.3.1 Feature Sketches

Having established design requirements, another iteration of sketches was initiated. The resulting sketches and an explanation of various functionalities are explained below.



Figure 4.10: Main menu with features

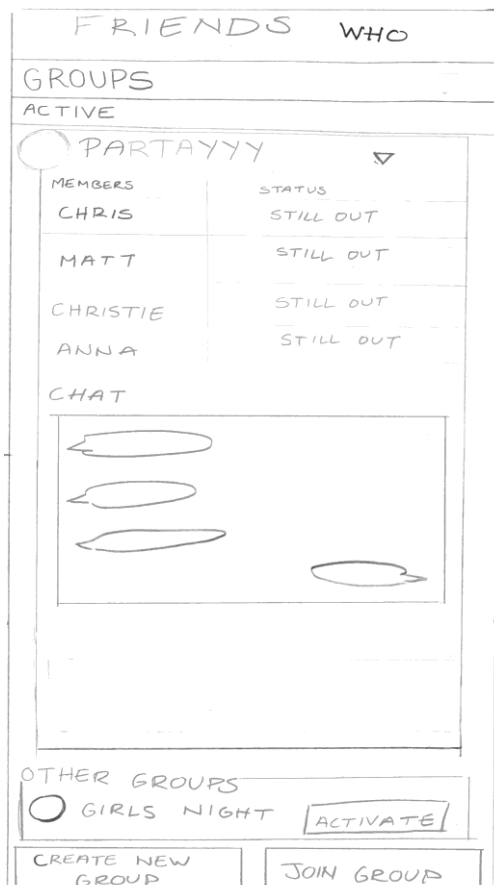


Figure 4.11: Groups page

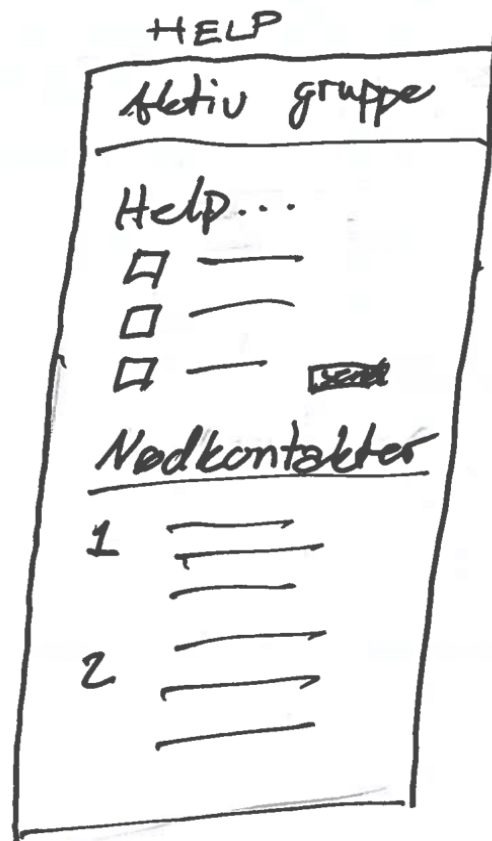


Figure 4.12: SOS page

Upon this iteration of sketches, the following four main features emerged:

- Notifications to tell friends that you got home
- Send an SOS message to emergency contacts
- Ability to locate yourself and friends on map
- Allow users to join and create groups with other people.

These features were categorized regards what functionalities are relevant within each feature, as seen in figure 4.10. The circles seen in this sketch are used as placeholders for icons that are designed later. Each circle corresponds with a nested page, and the four nested pages are: *Groups*, *Map*, *I'm Home* and *SOS*, see figure 4.10. Within the *Groups* page, see figure 4.11, it is possible chat with the user's friends in the activated group and see each friend's status. This feature respects the first and second requirements outlined in the Concept section. This functionality was also added based on our interviews, where we learned that people tend to not notice notifications from their friends, so we wanted this functionality to group relevant information from the user's friends on one page. On the *Map* page, a map, the user's and their friend's location are available. This feature was added to help the user find their friends, since we learned from our interviews, that people tend to lose their friends when partying, and we wanted this functionality to aid in finding one another again. On the *I'm Home* page, the user can mark

themselves as safe, subsequently turning off their GPS, so that they can no longer be tracked. This functionality was added to make it easier for the user to tell their friends that they have gotten home safe. As we learned in our interviews, people are concerned whether their friends get home safely, and if this is not communicated within 24 hours of the party, concern is risen. Lastly, on the SOS page, see figure 4.12, the user can request help from their friends. This was added to automate requests for help, rather than requiring the user to type it in the chat. The following section details our final sketches, derived from the sketches shown in this section.

4.3.2 Final Sketches

With another iteration of sketches completed, we discussed the features to be included, and, collectively, sketched another interface, that is the preliminary design we based our app upon. The final sketch can be seen below.

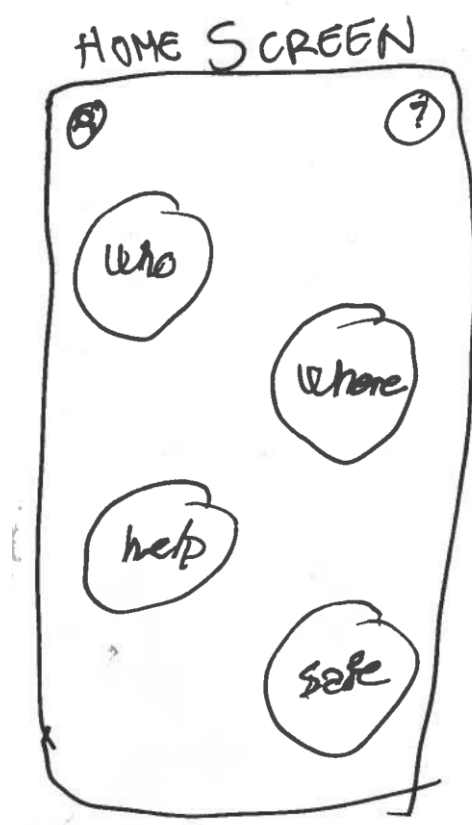


Figure 4.13: Final main menu

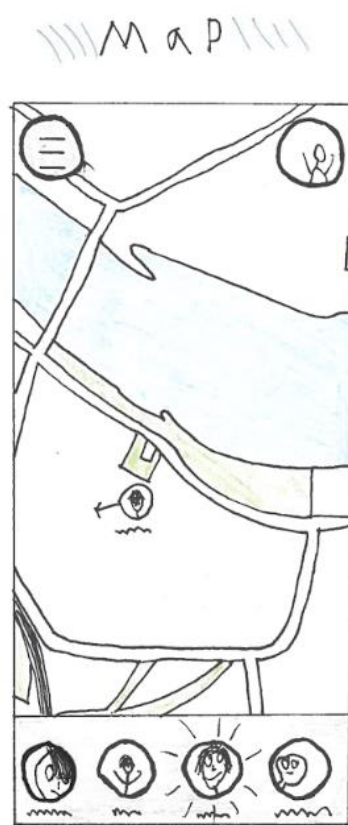


Figure 4.14: SOS page



Figure 4.15: Icons

With regards to this new main menu, as seen in figure 4.13, the functionality is largely the same as in the sketches previously detailed. When designing this sketch, we renamed the *Groups* page to *Who* and the *Map* page to *Where* to allow for future design alterations made to these functionalities, since we wanted to explore various features that could be included on the pages, even if it wasn't traditionally on pages associated with *Groups* or *Map*. The *Help* and *I'm Safe* feature were added to establish a safety net within the group. The *I'm Safe* feature functions as the *I'm Home* feature, but we didn't want the GPS to

only turn off once the user is home, since they might sleep elsewhere. Since they are still safe, regardless of where they sleep, we renamed this feature, so the user can activate it once they are safe rather than once they are home. This feature also removes *the Party Mode* as seen in previous sketches. By having a slider that turns the user's GPS on and off, mistakes are more likely to be made compared to having a button with the same functionality. A certain sense of urgency is related to the *Help/ SOS* feature, so we created dark scenarios to help us visualize how it could be misused or abused. The dark scenarios we came up with are:

- Using *Help/ SOS* feature to annoy friends
- If emergency contact list is accessible while the phone is locked, and someone finds the phone and calls friends as a prank.

By acknowledging these dark scenarios, we realized that certain features could become an annoyance, meaning that we needed to further develop these features. The main feature that could become an annoyance is the *Help* feature. This feature wasn't fully designed, and we needed to test the functionality of our app, so we omitted it from the Adobe XD prototype. To test our app, we developed a simple prototype in Adobe XD to test the locations and basic functionalities of the features of our design prior to coding it. This was done to see if potential users would understand the setup, rather than simply assume our design made sense to others, who hadn't been apart of our design process. This is detailed in the following section, alongside with our findings resulting from the test.

4.4 Adobe XD test

After having chosen our design, we wanted to test our concept with an Adobe XD prototype, as seen in figure 4.16. The functionality of the app was decided in the previously made sketches, but changes were made to the appearance of the elements. The design created for the test can be seen in figure 4.16 below.

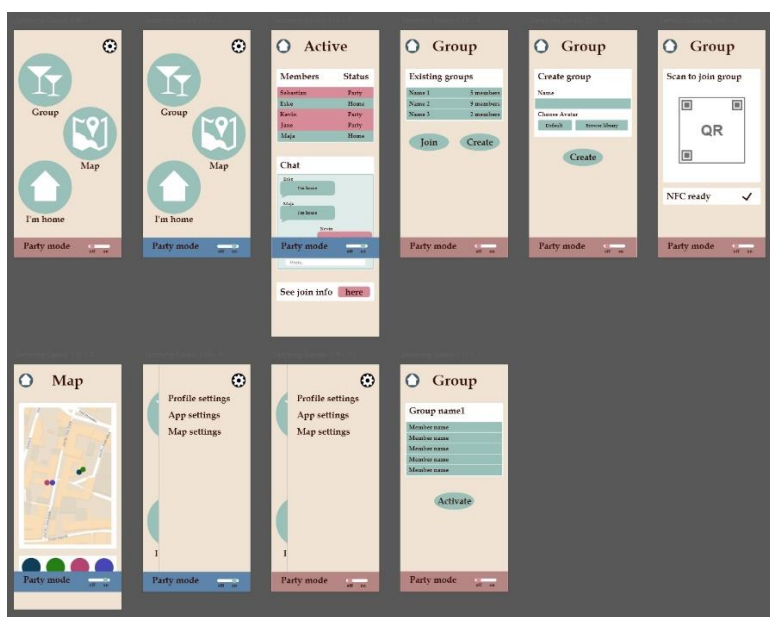


Figure 4.16: Adobe XD prototype

With a functioning prototype of the preliminary sketches, we could now loosely test our concept, specifically the placement of features and if this placement made sense. To test it, we gave the participant three scenarios they had to work through with the help of the prototype. The first scenario was used to see if the participant could create a group within the interface or if this didn't make sense. The scenario given to the participant, in Danish, but it has been translated for this report, was:

It is a Friday night and you met up with some friends prior to going to a party together. You have all downloaded this app, that helps you find each other and ensures that you've gotten home safely. You are ready before all your friends, so you make a new group that you all can be a part of. How would you create a new group and add your friends?

This scenario was solved without problem, illustrating that the simple nature of the *Groups* page aided in understanding how it is intended to be used. The second scenario regards the usage of the *Map* functionality. The second scenario given to the participant is:

You and your entire group of friends are now in Jomfru Ane Gade, and you started your night at Zwei Grosse. After a couple of beers, one of your friends needs a pack of cigarettes, and you go with them to 7 Eleven to buy a pack. You have all agreed to meet The Drunken Flamingo, but you can't find them again. How would you find your friends?

The participant opened the *Map* function instantly and saw that each group member is color coded, noting that the colors made it easier to know where every person was. When asked if the map would be enough to find her friends, the participant noted that the map was enough to find their general location outside, but it would not be of much help once she would need to find them inside. For the final scenario, we wanted to see if the *I'm Home* function made sense. The scenario given to the participant is:

The evening is ending, and you are all going home. You have all agreed to tell one another once you are all home, so that everyone can sleep comfortably in their own bed. You walk a friend to her home and then proceed home. How are you going to tell your friends that you are safely home?

The participant opened *I'm Home* immediately, saying that she would first activate *I'm Home* once she's gotten home. With this test being completed, we learned that each feature fulfilled their purpose. With this knowledge, we proceeded to create a further modified design of the app. The result is detailed in the following section.

4.5 Iterated Design

With our design chosen and tested, we made a version in Adobe Illustrator that we could base our code on, so that we all had a unified design to work from. The Adobe Illustrator figures can be seen below.

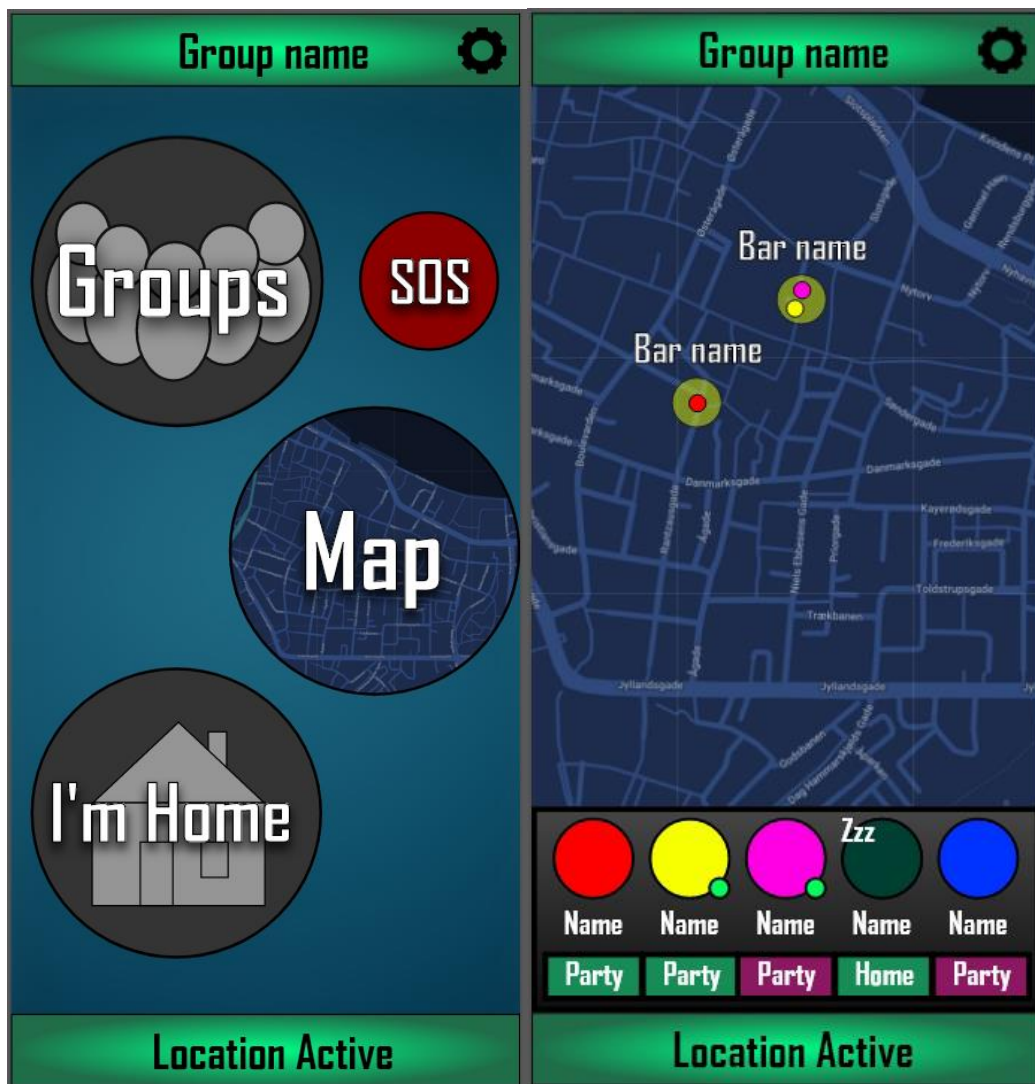


Figure 4.17: Modified main menu

Figure 4.18: Modified map page

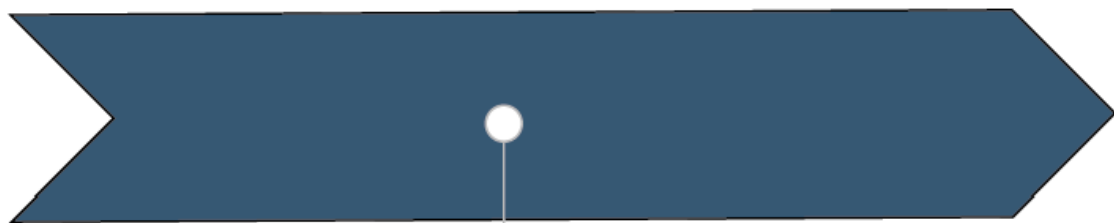
By finalizing the design, we brought into our coding process, we also defined the intended functioning of the *Help/ SOS* feature. We decided to name it *Help* to encourage inter-group communication, especially when the user is in danger. We have discussed including a feature that would contact the police, but, with the dark scenarios previously created, we realized that the possibility of contacting the police could be abused. Therefore, we removed this feature, and simply made a *Help* page that sends pre-made notifications to the group, if the user is feeling unsafe. With the design chosen and a prototype of the interface made, we proceeded to code our app, as detailed in the following chapter. In the section below, is a brief description of how we decided on the name for our app, which we carried into the Develop phase.

4.6 Naming the App

With our concept designed, we proceeded to name our app. Since the beginning of our design process, we referred to the app as "night-out", but it doesn't suggest the purpose of the app. With the distinction between making a party-focused app or a safety-focused app, we wanted to discuss potential names that would afford the focus of our app. Safety and connectedness are essential themes in our app, and we wanted the name to reflect that. With inspiration from the word safe and the cepheus star constellation, we came up with Sapheus. We morphed the two words together, further noting that the Saphe syllable looks like the word safe, since ph and f have similar sounds in English. With the name established, we proceeded to code our app, which is outlined in the next chapter.

5

Develop



Code

With the design and functionality decided, we moved into the second diamond of the Double Diamond, entering the Develop phase. In the Develop phase, "design led solutions are developed [and] iterated" (British Design Council, 2022b, p. 7). In this chapter, our coding process through the development of our app is detailed. First, general theory used for our interface design is detailed. Next, our code, essential functions and the resulting interface design are described. After each function is explained, the various notifications we used to provide the users with feedback are outlined to end this chapter.

5.1 Applicable Theory

This section details design theory applied in our interface design, as well as Jakob Nielsen's usability heuristic. We wanted to consider design theory and usability to make this iteration of our prototype as usable as possible, guiding certain design decisions.

5.1.1 Nielsen's Usability Heuristics

Nielsen's Usability Heuristics are "basic characteristics of usable interfaces... [that] apply to practically any type of user interface" (Nielsen, 1993, p. 115). By using usability heuristics, a heuristic evaluation can be conducted, with a goal of "[finding] the usability problems in a user interface design so that they can be attended to as part of an iterative design process" (Nielsen, 1993, p. 155). For the purposes of this chapter, six heuristics are applied, and are detailed below.

1. "Consistency: Users should not have to wonder whether different words, situations, or actions mean the same thing" (Nielsen, 1993, p. 20).
2. "Feedback: The system should always keep users informed about what is going on, through appropriate feedback within reasonable time" (Nielsen, 1993, p. 20).
3. "Clearly marked exits: Users often choose system functions by mistake and will need a clearly marked 'emergency exit' to leave the unwanted state without having to go through an extended dialogue" (Nielsen, 1993, p. 20).
4. "Shortcuts: Accelerators - unseen by the novice user - may often speed up the interaction for the expert user such that the system can cater to both inexperienced and experienced users" (Nielsen, 1993, p. 20).
5. "Good error messages: They should be expressed in plain language... precisely indicate the problem, and constructively suggest a solution" (Nielsen, 1993, p. 20).
6. "Prevent errors: Even better than good error messages is a careful design that prevents a problem from occurring in the first place" (Nielsen, 1993, p. 20).

As each heuristic is used in our design, it is explained alongside the resulting design. By keeping these heuristics in mind when coding and designing our app, we aim to minimize usability errors in our prototype. With the heuristics established, applicable design theory is detailed below and then how these theories are applied in our app are detailed underneath that.

5.1.2 Design Principles

To design our app interface, we relied on the following design principles to make our design pleasant for users: color theory and psychology, applicable Gestalt principles of Design and how affordance is applied to the interface. The aforementioned are detailed below and their appliance in our overall design is detailed in section 5.2.

Color Psychology

Color psychology is "the study of hues as a determinant of human behaviour. Most studies that centralize around color psychology focus on how a color is perceived by an individual or how it makes them feel" (Harris, 2018). The main colors throughout the design are blue, green, grey and white. We have decided to use a blue gradient as the background, since "[light] blue is associated with health, healing, tranquility, understanding, and softness. Dark blue represents knowledge, power, integrity, and seriousness" (Supercolor, 2020), all feelings we wanted to promote when opening Sapheus. To provide feedback to users, as specified by Nielsen's heuristics (p. 20), we used the color green, since it "is associated with refreshment and peace, rest and security" (Wharton, 2019). We used green to signal that the user can go further, as afforded from traffic lights (Butler et al., 2010, p. 22).

Gestalt Principles

Events trigger a code changing the color of the object to green, providing the user with direct feedback from their interaction. To display the difference between click-able icons and the background, we used Gestalt's figure-ground principle. The figure-ground principle states that "[elements] are perceived as either figures (objects of focus) or ground (the rest of the perceptual field)" (Butler et al., 2010, p. 96). We have used the colors white and grey to achieve this distinction between figures and the background. By consistently using white and grey to distinguish figure-ground, we accommodate the "Consistency" heuristic (Nielsen, 1993, p. 20). An additional Gestalt principle applied in our design is similarity, which states that "[elements] that are similar are perceived to be more related than elements that are dissimilar" (Butler et al., 2010, p. 226). This is applied on the main page that accesses all of the subsequent pages, since all the icons have the same size, they are all viewed as related. With relevant design theory detailed, our code and resulting app are explained in the following section, alongside the specific application of the principles mentioned in this chapter.

5.2 Code

This section details important functions of the app, explains the underlying code and the resulting interface design. We chose to develop our app in Android Studio, which uses Java as the programming language and XML for assembling the layout and design, and shared code through GitHub. The full code of the application can be found in the Appendix since this section solely details important functions in Sapheus.

Shared Data

SharedData, see figure 5.1, is a Java class central to the functionality of Sapheus. It uses static variables, a variable type that only exists once, regardless of how many times the class is initialized. This makes the variables shareable among all other objects of the same class or any other class. Its main use is to keep track of the *arraylists* that contain all the

information about friends that the user has on the app. In the *SharedData* class, *ArrayLists* are used instead of regular arrays, since *arraylists*, in Java, are dynamic and can change in size. (Baeldung, 2021)

```
15      // Your friends stored data
16      public static ArrayList<String> FriendNames = new ArrayList<>();
17      public static ArrayList<String> FriendStatus = new ArrayList<>();
18      public static ArrayList<LatLng> FriendPositions = new ArrayList<>();
19      public static ArrayList<BitmapDescriptor> FriendIcons = new ArrayList<>();
20      public static ArrayList<String> FriendLastPing = new ArrayList<>();
21
22      // Stored group data
23      public static ArrayList<String> GroupNameArray = new ArrayList<>();
24      public static ArrayList<String> GroupMemberCountArray = new ArrayList<>();
25      public static int NumberOfGroups = 0;
26
27      // Active group ID
28      public static int ActiveGroup = 0;
```

Figure 5.1: Code snippet of *SharedData.java*

SharedData is a class that collects important variable throughout the app. We had chosen to collect the variable into a class grouping them, to make them more changeable rather than the need of searching for each variable on each page, if changed were made to them. In collaboration with this, we also used the built-in *SharedPreferences* function. This function stores data locally on Android devices, through a file created in the app written to the device. This function is used to store important data, that the users need to interact with the app. This makes it possible to save login information, so the users remain logged in when they open the app, and this makes it possible to store information that can be used between different functions in the app.

```
55      //SharedPreferences initiated copy paste the line below if you need the preference in an activity
56      SharedPreferences pref = getSharedPreferences( name: "com.android.SafeNight.settings", Context.MODE_PRIVATE);
57
58      //SharedPreferences editor to create and edit variables
59      SharedPreferences.Editor editor = pref.edit();
60      editor.putBoolean( s: "MapToggle", b: false);
61
62      editor.apply();
```

Figure 5.2: Code snippet using *SharedPreferences*

Storing data is split into sub functions. To start, initialize the creation of the *preference* file through this command, see figure 5.2, line 56. Here, we set the name for the variable to call the *preference* file, then save the file *com.android.SafeNight.settings* and set the mode to private, so the file only is accessible in the app and can't be altered or overridden by other apps. Secondly, the file is made editable and writeable, see line 59. This is done by taking the written code that can be seen in the first step and appending the *.edit()* function, built in the *SharedPreference* class, which stores the file in a variable that is used

to call the *editor*, as seen on line 62. Next, the variables from the prior step are defined, see figure 5.3, line 210. Here, the *editor* variable created in the second step is called with a *.put* ending, with the datatype specified next to it, to define which type of variable is being stored. Inside the parentheses, the name and value of the variable are stored.

```
215 editor.putString( s: "AddressName", AddressText.getText().toString());
216 editor.putString( s: "EmergencyName1", EmergencyName1.getText().toString());
217 editor.putString( s: "EmergencyNumber1", EmergencyNumber1.getText().toString());
218 editor.putString( s: "EmergencyName2", EmergencyName2.getText().toString());
219 editor.putString( s: "EmergencyNumber2", EmergencyNumber2.getText().toString());
220 editor.putString( s: "EmergencyName3", EmergencyName3.getText().toString());
221 editor.putString( s: "EmergencyNumber3", EmergencyNumber3.getText().toString());
222 editor.apply();
```

Figure 5.3: Code snippet of *ProfilePage.java*

The last step is using the *apply* function, which can be seen on line 222. It is used on the previously created *editor* variable, to save new or changed variables to the file.

```
129 // Load the emergency contacts
130 EmergencyName1.setText(pref.getString( s: "EmergencyName1", s1: ""));
131 EmergencyNumber1.setText(pref.getString( s: "EmergencyNumber1", s1: ""));
132 EmergencyName2.setText(pref.getString( s: "EmergencyName2", s1: ""));
133 EmergencyNumber2.setText(pref.getString( s: "EmergencyNumber2", s1: ""));
134 EmergencyName3.setText(pref.getString( s: "EmergencyName3", s1: ""));
135 EmergencyNumber3.setText(pref.getString( s: "EmergencyNumber3", s1: ""));
```

Figure 5.4: Code snippet of *HelpPage.java*

To read the data from the *SharedPreferences* file, see figure 5.3, a *get* function with the requested datatype is needed, see line 130. Inside the parenthesis, the function needs a variable to search for and beside a placeholder variable if the variable in the *preference* file is empty. We used the *SharedPreferences* function to improve the user experience, by skipping login, linking other functions, and getting necessary information for grouping users related ones to achieve a feeling of connectedness. The usage of these functions resulted in the functioning of the following pages.

Profile Page

The profile page, see figure 5.5a, is where the user can see their information and edit them if needed. The page is made of text boxes that shows *their full name, address, number, and emergency contacts*. At the bottom, an *edit* button is placed to make the information editable if needed. By making the functions, it enabled us to make a profile page where users can save necessary information, so that the app can function as intended. Since GPS signals are used to help friends see each on the app, the ability to save information, like one's address, will automate functionalities that are necessary for the functioning of the app. The following figure shows the preliminary setup of the profile page.

Figure 5.5: Profile page when not editing

Figure 5.6: Profile page when editing

The XML code in this page, uses special *EditText* boxes, which be seen in figure 5.7. These text boxes accept user input. In the *EditText* boxes, the input type can be set, which we defined so the user can't insert illegal characters in certain text fields, such as letters in the phone numbers or numbers in names. The *android:hint* is used as placeholder to suggest the type of expected information. *Android:enabled* is set to false, so the text isn't editable as default when the page is entered until the edit button is pressed. The *EditText* boxes are used for exporting user information into the *SharedPreferences* file in the Java code.

```

68      <EditText
69          android:id="@+id/FullNameText"
70          android:layout_width="350dp"
71          android:layout_height="37dp"
72          android:enabled="false"
73          android:hint="Empty"
74          android:inputType="textPersonName"
75          android:lineSpacingExtra="8sp"
76          android:textColor="#FFFFFF"
77          android:textColorHint="#818181"
78          android:textSize="16sp"
79          app:layout_constraintBottom_toBottomOf="parent"
80          app:layout_constraintEnd_toEndOf="parent"
81          app:layout_constraintHorizontal_bias="0.508"
82          app:layout_constraintStart_toStartOf="parent"
83          app:layout_constraintTop_toTopOf="parent"
84          app:layout_constraintVertical_bias="0.149" />

```

Figure 5.7: Screenshot of the Profile page XML.

When the *edit* button is pressed, the *SharedData editmode* variable is changed to true and triggers the code snippet seen in figure 5.8 that makes all the textboxes editable by changing the *android:enabled* to true for each box. The *edit* button's text becomes an *apply* button, to implore the user to save their altered information, which can be seen in figure 5.6.

```

132      public void openProfileEdit() {
133
134          // if editmode is true, then enable all edittext to be edited
135          if (SharedData.EditMode == true) {
136              FullNameText.setEnabled(true);
137              MobileText.setEnabled(true);
138              EmailText.setEnabled(true);
139              PostNumberText.setEnabled(true);
140              CityText.setEnabled(true);
141              AddressText.setEnabled(true);
142              EmergencyName1.setEnabled(true);
143              EmergencyNumber1.setEnabled(true);
144              EmergencyName2.setEnabled(true);
145              EmergencyNumber2.setEnabled(true);
146              EmergencyName3.setEnabled(true);
147              EmergencyNumber3.setEnabled(true);

```

Figure 5.8: Screenshot of the profile page Java.

When the user has filled in his or her information and has pressed the *apply* button, the code snippet, seen in figure 5.9 is initiated. This code validates the information in the *EditText* boxes. For example, it checks if the e-mail contains "@" and ".", using the *contains* function built into *Android Studio* and if the length is longer than 0. Once validated, the code saves the information in the *SharedPreferences* file. Once code is executed, the *apply* button is set back to the *edit* button.

```

283 // if email has "@" and "." in it or is empty
284 if (EmailText.getText().toString().contains("@") && EmailText.getText().toString().contains(".")
285     || EmailText.getText().toString().length() == 0) {
286
287     // save the data to pref
288     SharedPreferences pref = getSharedPreferences("com.android.SafeNight.settings", Context.MODE_PRIVATE);
289     SharedPreferences.Editor editor = pref.edit();
290     editor.putString("FullName", FullNameText.getText().toString());
291     editor.putString("Number", MobileText.getText().toString());
292     editor.putString("Email", EmailText.getText().toString());
293     editor.putString("PostNumber", PostNumberText.getText().toString());
294     editor.putString("City", CityText.getText().toString());
295     editor.putString("AddressName", AddressText.getText().toString());
296     editor.putString("EmergencyName1", EmergencyName1.getText().toString());
297     editor.putString("EmergencyNumber1", EmergencyNumber1.getText().toString());
298     editor.putString("EmergencyName2", EmergencyName2.getText().toString());
299     editor.putString("EmergencyNumber2", EmergencyNumber2.getText().toString());
300     editor.putString("EmergencyName3", EmergencyName3.getText().toString());
301     editor.putString("EmergencyNumber3", EmergencyNumber3.getText().toString());
302     editor.apply();

```

Figure 5.9: Screenshot of the Profile page Java.

By coding a profile page, we can use the user's input, through the *SharedPreferences* function, to further establish a safety net between friends, and the *emergency contacts* feature was added to allow people outside the group to receive the messages from the user that has specified the contact. The code previously detailed has resulted in the following design of the profile page.

Figure 5.10: Not editing.

Figure 5.11: When editing

As previously detailed, the profile page has two buttons and text boxes used for user input regarding their personal information, the design can be seen in figure 5.10. When the *edit* button is clicked, the background color of the text boxes changes to a darker grey, signalling that the text box is editable, respecting the "Feedback" heuristic (Nielsen, 1993, p. 20). Next, the main menu is detailed.

Main Menu

The main menu's basic setup can be seen in figures 5.12 and 5.13.

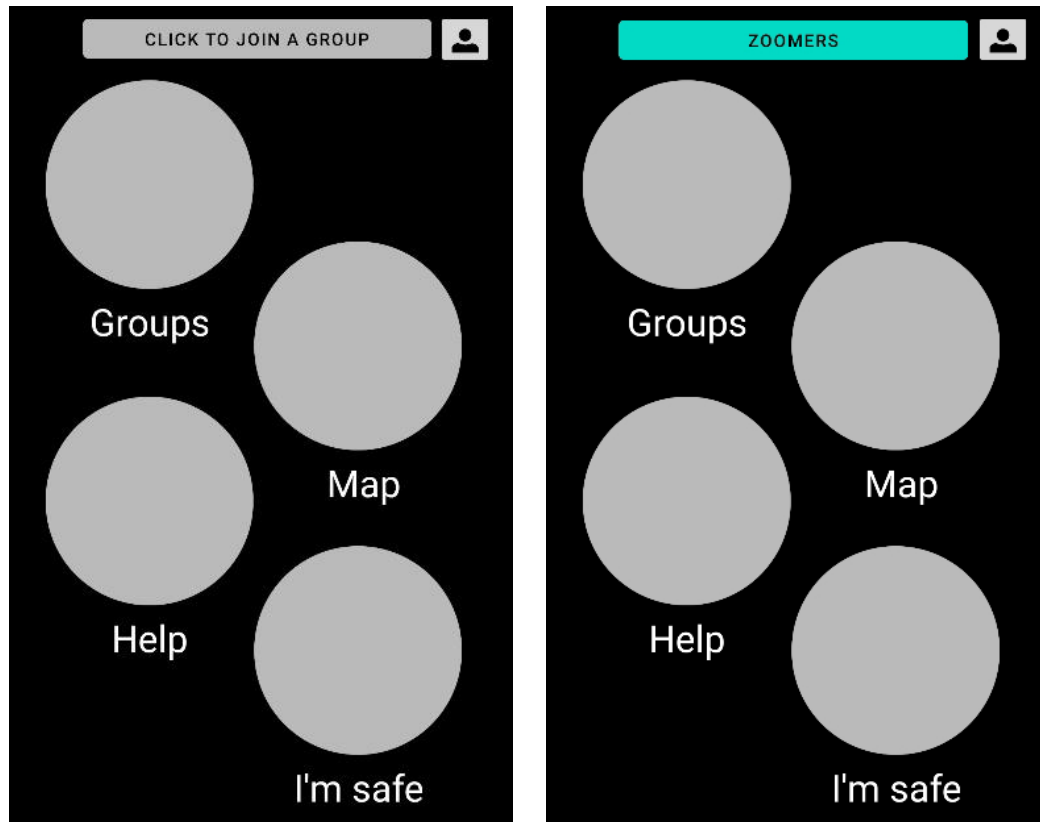


Figure 5.12: Main menu when not in a group. Figure 5.13: Main menu when in a group

At the top of figure 5.12, the *SharedData* class is implemented, seen in the group activation bar. On line 28 in figure 5.1, the *int ActiveGroup* is created and assigned to 0. This variable is used to keep track of what group is activated. When it is 0, the other parts of the app know that the user hasn't activated a group, and when the *int* is set to 1, for example, then the *SharedData* function knows the user has activated group 1, showing the user what group is active, regardless of what page they are on, as seen in figure 5.13.

The main menu is the central hub of the app, see figures 5.12 and 5.13, where the user can click on various buttons to access the functionalities offered in the app. The *Groups* button accesses the accompanying page, where the user can see their friend's status and chat within the group. The *Map* button accesses Google Maps, where the user can see their own and their friends' current locations. The *Help* button leads to the accompanying page, where users can request help from their friends and emergency

contacts. By long pressing the *I'm Safe* button, the user's status is marked as *Home* and the user's GPS signal is turned off. As previously mentioned, long-pressing is required to activate *I'm Safe*, so it cannot be activated by mistake, respecting the "Preventing errors" heuristic (Nielsen, 1993, p. 20). The XML code in figure 5.14, shows the code used to make the four buttons.

```

190      <ImageButton
191          android:id="@+id/buttonGroups"
192          android:layout_width="123dp"
193          android:layout_height="175dp"
194          android:scaleType="fitCenter"
195          app:layout_constraintBottom_toBottomOf="parent"
196          app:layout_constraintEnd_toEndOf="parent"
197          app:layout_constraintHorizontal_bias="0.178"
198          app:layout_constraintStart_toStartOf="parent"
199          app:layout_constraintTop_toTopOf="parent"
200          app:layout_constraintVertical_bias="0.203" />

```

Figure 5.14: Code snippet from *main_menu.xml*.

This XML code snippet in figure 5.14, shows how the button *Groups* is made. It is an *ImageButton* with the ID *buttonGroups*, which is located in the Java code. The images the button uses is then changed through the Java code, depending on the state of the button. When the button is pressed, it goes green for a bit, to show that the button was pressed, respecting the "Feedback" heuristic (Nielsen, 1993, p. 20), since it provides feedback regarding the status of the system in processing the request from the user. The final design of the main menu can be seen in figures 5.15, 5.16 and 5.17.

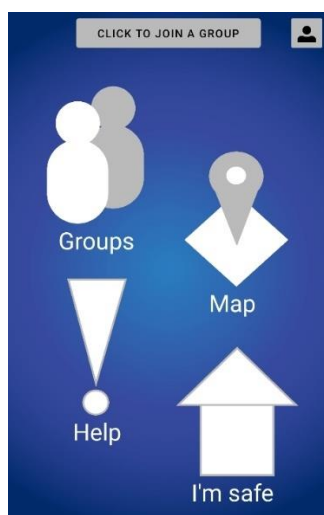


Figure 5.15: main menu,
No group active

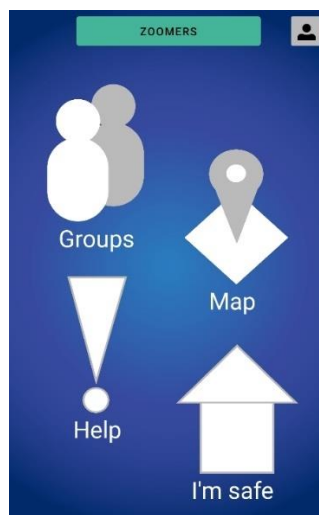


Figure 5.16: main menu, group
active



Figure 5.17: main menu, group
and I'm safe active

The main menu, as seen in figure 5.15, consist of icons that are linked to the pages previously described. The icons were designed based on the overall purpose of the function, affording what the user can expect to find if they choose to click on that button. The green *Active Group* bar at the top of the screen serves, as seen in figure 5.16 as a shortcut for the user to access the *Groups* page, respecting the "Shortcuts" heuristic (Nielsen, 1993, p. 20). The *I'm Safe* icon, as seen in figure 5.17 is a button used to confirms to the group, that the user has got home safely. The icon turns green to signal that the group has been notified, respecting the "Feedback" heuristic (Nielsen, 1993, p. 20). The code for each of the functionalities are detailed below, starting with the *Groups* function, then, the *Maps* function and lastly the *Help* function.

Groups Page

The *Groups* page has two states, when the user is in an active group or one where the user is prompted to activate or join a group. When the user is not in a group, figure 5.18 appears. If the user is in an active group, figure 5.19 appears.

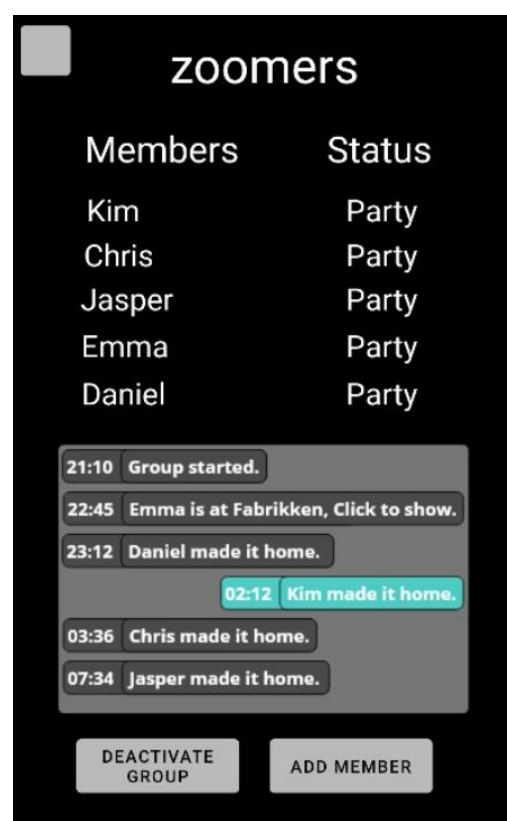


Figure 5.18: Groups page when not in a group. Figure 5.19: Groups page when in a group.

The following code was used to make the *Groups* page.

```

71      // Loads data of the active group
72      public void ActiveGroupFunc() {
73          SharedData.ActiveGroupFriendIDs.add(0); // Add yourself first, you are ID 0
74          if (SharedData.ActiveGroup == 1) {
75              SharedData.ActiveGroupFriendIDs.add(5);
76              SharedData.ActiveGroupFriendIDs.add(4);
77              SharedData.ActiveGroupFriendIDs.add(1);
78              SharedData.ActiveGroupFriendIDs.add(2);
79          }
80          if (SharedData.ActiveGroup == 2) {
81              SharedData.ActiveGroupFriendIDs.add(4);
82              SharedData.ActiveGroupFriendIDs.add(2);
83              SharedData.ActiveGroupFriendIDs.add(9); // 9 equals empty
84              SharedData.ActiveGroupFriendIDs.add(9); // 9 equals empty
85          }

```

Figure 5.20: Screenshot of the Group page Java.

In figure 5.20, the function on line 72 is initialized when the *GroupActivePage* is loaded. It checks what group the user is in and adds the correct group members to the group. The first three groups are pre-made examples, which is why it has specific members that join it. In line 83 and 84, the group member with an ID of 9 is added, which is an empty friend, who doesn't have a name and their location is 0.0. This is recognized by other parts of the app, that is set to ignore a friend and set the slot as empty, when their ID is 9.

```

123      FriendNameDisplay1.setText(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(0)));
124      FriendNameDisplay2.setText(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(1)));
125      FriendNameDisplay3.setText(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(2)));
126      FriendNameDisplay4.setText(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(3)));
127      FriendNameDisplay5.setText(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(4)));
128      Status1.setText(SharedData.FriendStatus.get(SharedData.ActiveGroupFriendIDs.get(0)));
129      Status2.setText(SharedData.FriendStatus.get(SharedData.ActiveGroupFriendIDs.get(1)));
130      Status3.setText(SharedData.FriendStatus.get(SharedData.ActiveGroupFriendIDs.get(2)));
131      Status4.setText(SharedData.FriendStatus.get(SharedData.ActiveGroupFriendIDs.get(3)));
132      Status5.setText(SharedData.FriendStatus.get(SharedData.ActiveGroupFriendIDs.get(4)));

```

Figure 5.21: Screenshot of the Group page Java, loading friends.

In figure 5.21 on line 123, the name of the first friend is loaded. The textbox is set to the first friend in the *FriendNames* arraylist. The group page is the main feature of the app. The feature shares each user's name and collects them in a group. We have chosen to explain this function because the app wouldn't be functional without it. The *Group status bar* feature was included in the final design to improve the navigation inside the app by having a shortcut and a way to give feedback. It signals to the user that it's connected to a group and aids the feeling of connectedness.

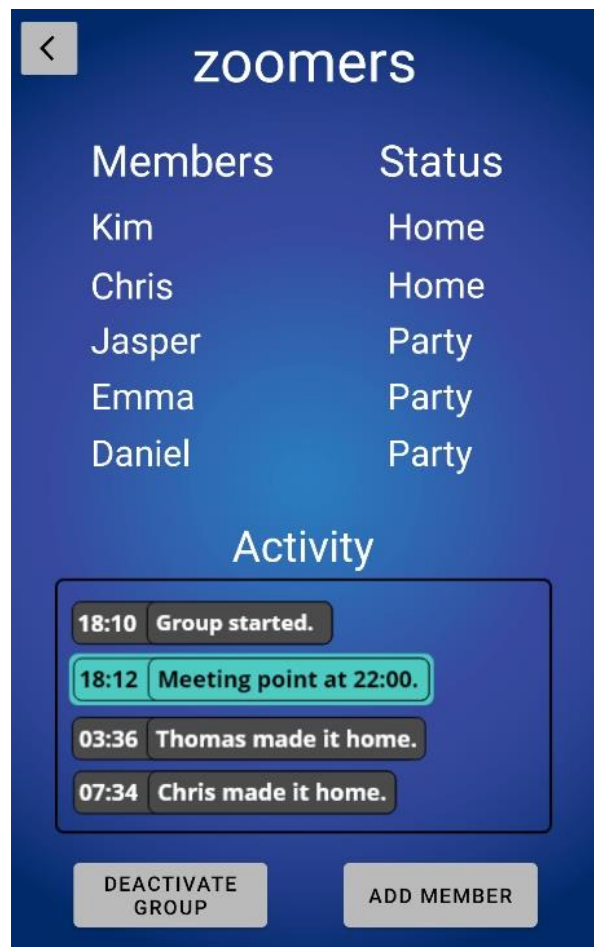
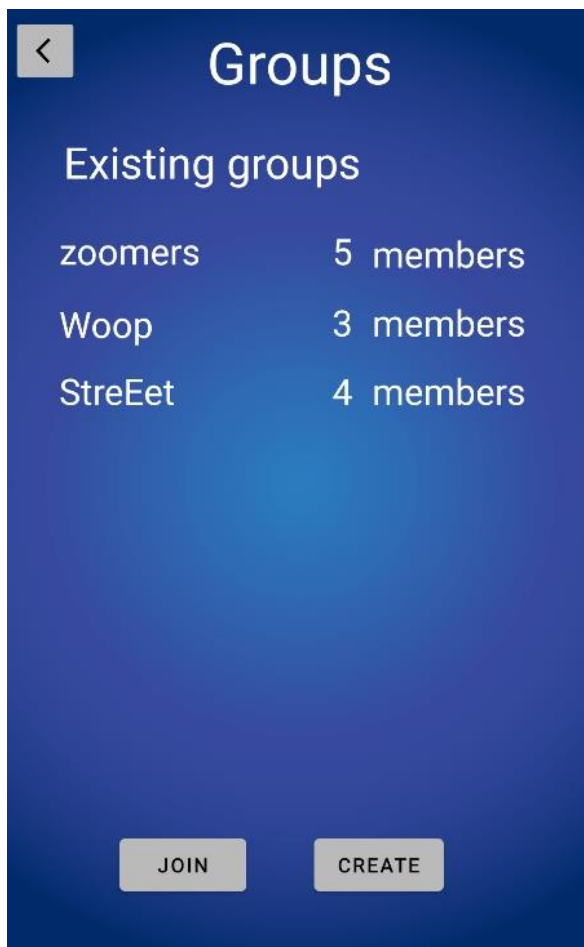


Figure 5.22: Screenshot of final Groups page

Figure 5.23: Screenshot of final active Groups page

The other part of the *Groups* function shows the status of group members to add another communication feature that increases the available information of each users to ensure that the available information enhances the feeling of safety and connectedness. All information regarding the group is available on this page. With the *Groups* feature explained, the *Map* feature is detailed below.

Map

The *Map* is another main feature in Sapheus and can be seen in figure 5.26. The map allows the user to locate their friends, and to establish meeting point to meet at. This feature was included based on the interviews conducted in the Define phase, where we learned that people tend to lose their friends during a party. The following code makes the map feature functional.

```

193 // Zoom in to Aalborg
194 LatLng aalborg = new LatLng( latitude: 57.0470642, longitude: 9.9202487);
195 float zoomLevel = 14.0f; //This goes up to 21
196 mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(aalborg, zoomLevel)); //from: https://stacko
197 Log.d( tag: "Test", msg: "OnMapReady");
198 mMap.setOnMapLoadedCallback(this);
199
200 // check if location permission is granted and then get location
201 if (SharedData.UseRealLocation == true) {
202     if (ActivityCompat.checkSelfPermission( context: MapPage.this,
203         Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED) {
204         getLocation();
205     } else {
206         ActivityCompat.requestPermissions( activity: MapPage.this,
207             new String[]{Manifest.permission.ACCESS_FINE_LOCATION}, requestCode: 44);
208     }
209 }
210 // Set group name
211 ShowActiveGroupName();
212 }

```

Figure 5.24: Usage of SharedData in MapsPage.Java

On line 194 in figure 5.24, the location "Aalborg" is set, since this is our relative location, making it easier to test the functionality of the map. It is a *LatLng* coordinate, which is longitude and latitude that works with Google Maps that has been imported as *mMap*. On line 202, the code checks if the app has been given access to the user's location, and requests it, if needed. If access is granted, the code calls *getLocation*, which gets the current, real GPS location of the user. With the code related to position explained, the following code was used to make the markers that depict friends' location on the map.

```

336 for (int i = 0; i < NumberOfFriends; i++) {
337     if (SharedData.FriendStatus.get(SharedData.ActiveGroupFriendIDs.get(i)).equals("Party")) {
338         Friendsmarker.add(mMap.addMarker(new MarkerOptions().
339             position(SharedData.FriendPositions.get(SharedData.ActiveGroupFriendIDs.get(i))).
340             title(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(i))).
341             snippet(SharedData.FriendLastPing.get(SharedData.ActiveGroupFriendIDs.get(i))).
342             icon(SharedData.FriendIcons.get(i))));
343     }
344 }
345 }

```

Figure 5.25: Code snippet of MapsPage.Java

The code snippet seen in figure 5.18 puts markers on the map for each friend. The code starts at line 336 by running a *for-loop* conditional to the number of members in the group. On line 337, the code checks if a friend is marked as *Party* and won't run the rest if the friend is marked as *Home*. On line 338, the code adds a marker to *mMap* and gets the position of the friend from the *arraylist* of friend positions in *SharedData*. This loads the friend's position depending on what unique ID's the active friends have. It does the same with the title, snippet and icon. The snippet is the description of the marker, which is found beneath the title. This loop runs every time the map is updated, so that the position of friends can change. Before it puts down the new markers, the code also runs *mMap.clear*, which clears the map of all current markers, so new ones can be placed in new positions.

The *Map* feature was included to prevent more cases like Oliver's from happening again and establishing a feeling of connectedness and safety among friends. The meeting point feature was designed to give the users a way to share a location where they want to meet. To create a meeting point, a long press is required, and a box appears to confirm the creation of a meeting point. This respects the "Preventing errors" heuristic (Nielsen, 1993, p. 20). Meeting points allow users to feel connected while using the app, since they can always meet up again, should they lose one another. On the map in figure 5.26, the user can click on the *show friends'* button, to get a color-coded list of their friends, corresponding to the colored dot on the map. If friends have marked themselves as *Home*, then their names are grey, and no dot is visible on the map. The user can click on their names to quickly move the map to that friend's location. When clicking on a friend on the map, it shows their name and their last ping, in case their connection is lost.

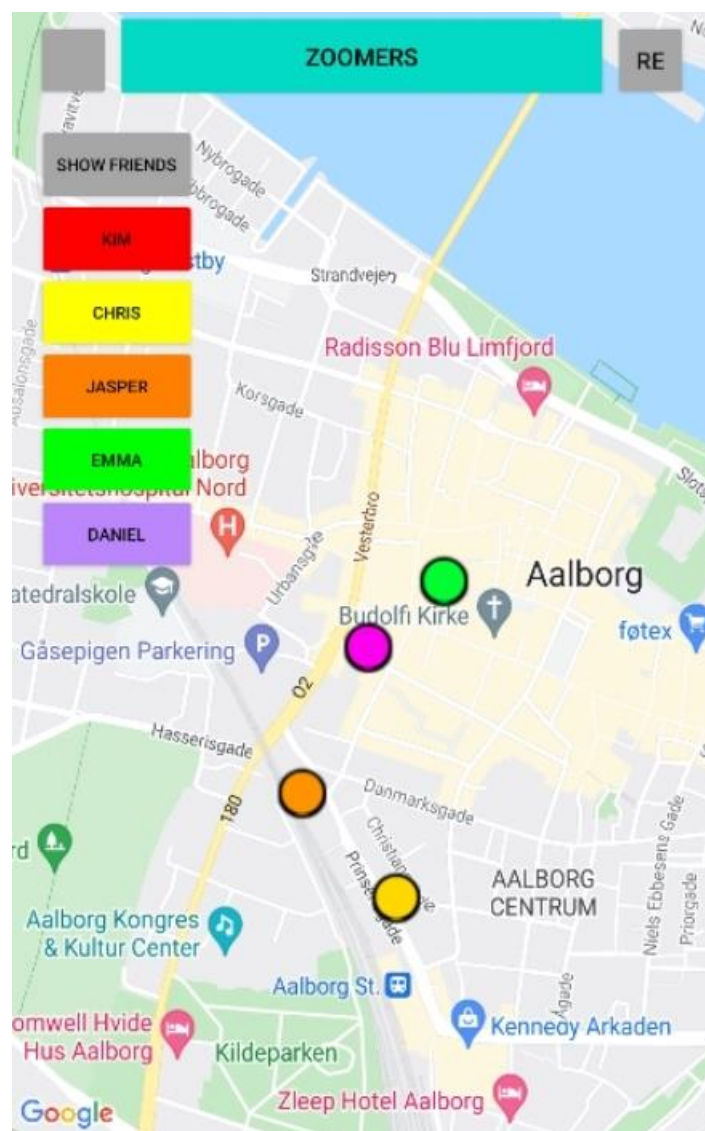


Figure 5.26: Screenshot of Map feature

With the functioning of the *Map* detailed, the design can be seen in figures 5.27 and 5.28.

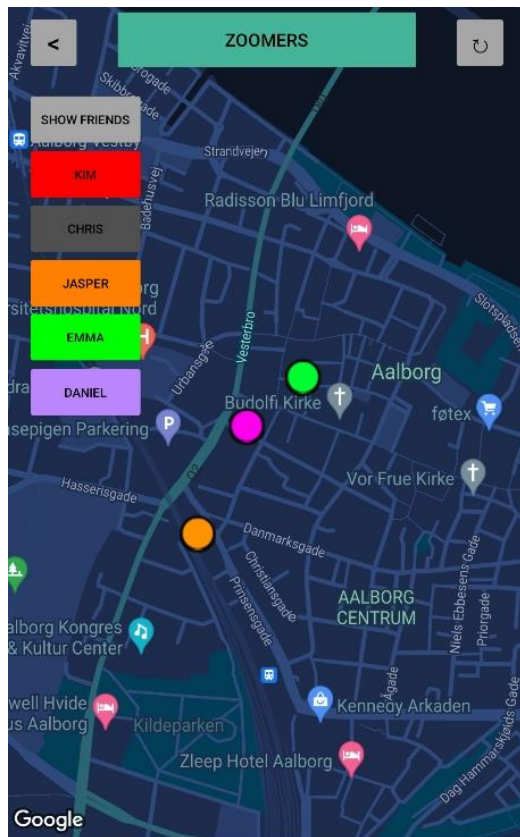


Figure 5.27: Screenshot of final map design

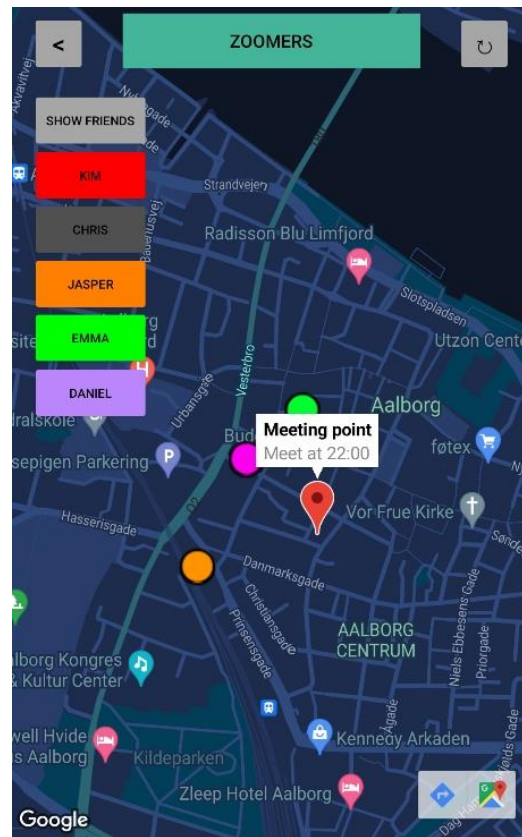


Figure 5.28: Map with a meeting point created

Help

The *Help* page as seen in figure 5.29, is designed for emergencies. There are two buttons to send requests for help to the user's emergency contacts and the friends in the active group that the user is in. The design includes two pre-made buttons to make it easier for the user to request help, should the user need it. The two buttons are *HELP ME!* and *I'M UNSAFE*. Underneath the buttons, is a list of emergency contacts, so the user can see what emergency contacts that were added in case the user forgets. The emergency contact information is loaded to this page via the *SharedPreference* file, that is edited in the profile page. There's an *add contact* button at the bottom of the page, that sends the user to the profile page. The *Help* page was added to facilitate a layer of communication between friends. This was done through notifications to tell the users, that their message was sent to their friends. With the pre-made buttons, the emergency contacts and using notifications to provide feedback, we aimed to increase the connectedness of the users.

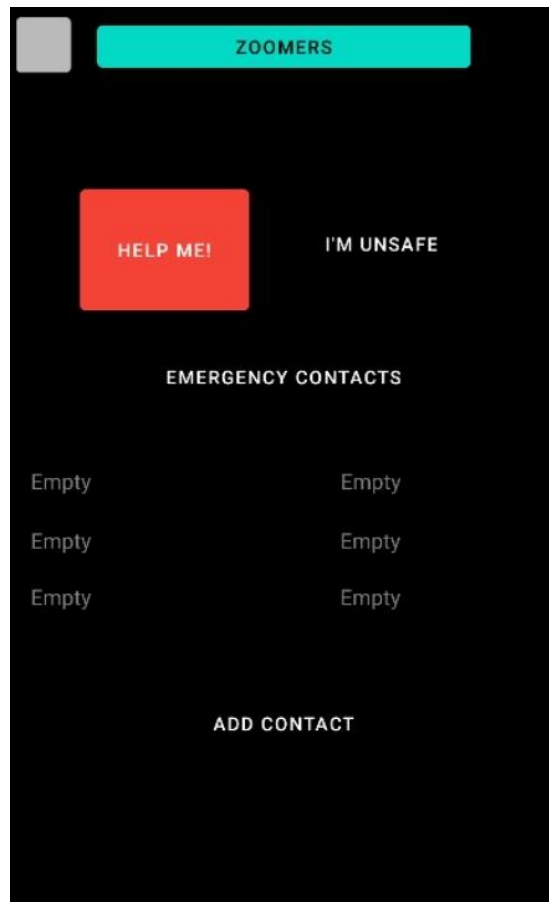


Figure 5.29: Screenshot of Help page

In figure 5.30, the used Java code to create a notification when the *Help Me!* Button is clicked. Since a server hasn't been established yet, we set the notification to send to the user, to illustrate that the system is responding to the help request. Once a server and database are established, this code can be changed to send the notification to the group rather than to the user themselves.

```
if(Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    NotificationChannel channel = new NotificationChannel( id: "HelpMeNotification", name: "help me notification", NotificationManager.IMPORTANCE_DEFAULT);
    NotificationManager manager = getSystemService(NotificationManager.class);
    manager.createNotificationChannel(channel);
}

//notification code, makes and sends notification
HelpMeButton.setOnClickListener((View v) -> {
    NotificationCompat.Builder builder = new NotificationCompat.Builder( context: HelpPage.this, channelId: "HelpMeNotification") //channelid == id from
        .setContentTitle("Notification Sent")
        .setContentText("Help Me notification sent!")
        .setSmallIcon(R.drawable.positionicon4)
        .setPriority(NotificationCompat.PRIORITY_DEFAULT);

    NotificationManagerCompat managerCompat = NotificationManagerCompat.from(HelpPage.this);
    managerCompat.notify( id: 20, builder.build());
});
```

Figure 5.30: Code snippet creating a notification when Help Me! Button is clicked

The final design of the *Help* page can be seen in figure 5.31 below.

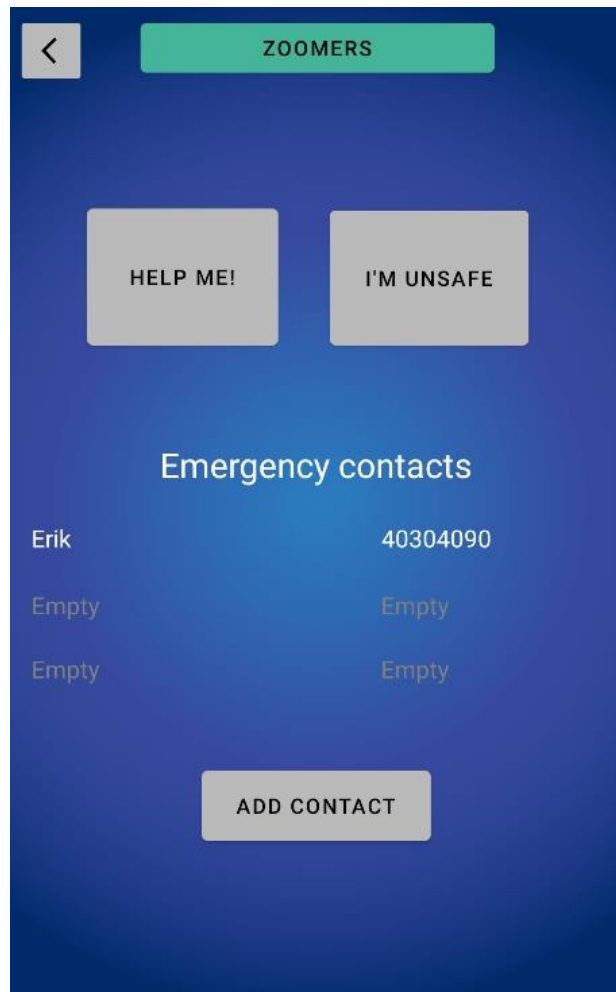


Figure 5.31: Final help page design

In our design, we have used various notifications to improve the feedback from the app to the user. The types of notifications used in Sapheus are detailed below.

Notification Types

There are several ways to create notifications in Android Studio. Sapheus utilizes toasts, heads-up and regular notifications, figures 5.32, 5.33, and 5.34 show examples of the notifications.

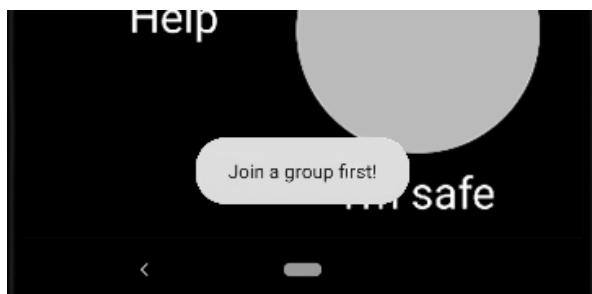


Figure 5.32: Toast notification

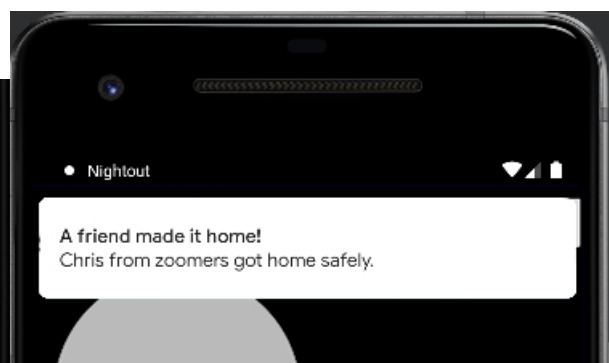


Figure 5.33: Heads-up notification

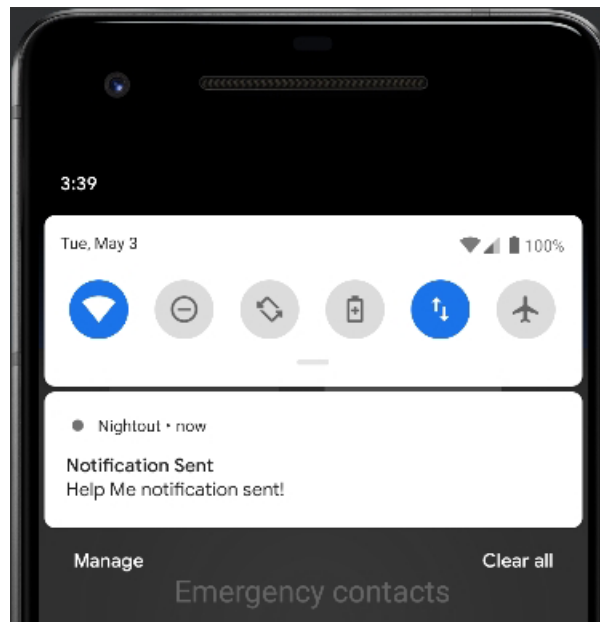


Figure 5.34: Regular notification

The first figure, 5.32, is a toast notification, which appears at the bottom of the app and disappears after a few seconds. We used toasts immediately after a user action, to provide feedback to the user. This respects the heuristics of "Feedback...[and] Good error messages" (Nielsen, 1993, p. 20), as the user gets notified of the systems status and relevant error messages. The following code snippet shows how we coded a toast notification.

```

134      ImSafeButton.setOnClickListener(v -> {
135          if (SharedData.ActiveGroup == 0) {
136              // toast notification
137              Context context = getApplicationContext();
138              CharSequence text = "Join a group first!";
139              int duration = Toast.LENGTH_SHORT;
140              Toast toast = Toast.makeText(context, text, duration);
141              toast.show();
142          }

```

Figure 5.35: Code snippet from MainMenu.Java that makes the toast notification

In Sapheus, a toast notification appears when *the I'm Safe* button is clicked on the *Help* page, line 134 and when the user has not activated a group, line 135. The first step of making the toast notification is to initiate an object to the *Context* class, that is called context. On line 138, the object text is initiated to the *CharSequence* class and it is set to *Join a group first!*. On line 139, the length of the notification is set, which are fixed lengths with the options of *SHORT* and *LONG*. The last part of the code on line 140, takes the aforementioned functions and combines them into a toast object, which is then shown on line 141 with the *toast.show()*. The second figure, 5.33, is a heads-up notification, which creates a pop-up notification, like a toast notification, which disappears, but at the top of the app. This heads-up notification is shown when a friend makes it home, because this

is an important event that needs to be break the "Consistency" (Nielsen, 1993, p. 20) the user is accustomed to from the toast notifications that appear more frequently. The third figure, 5.34, is a regular notification, which doesn't create a pop-up at the top of the screen, it simply creates an icon notification at the top of the screen, along with the WiFi and the battery icons. This notification can be read by pulling down the menu. The heads-up notification in figure 5.33, can also be read here after it disappears and is shown as a regular notification.

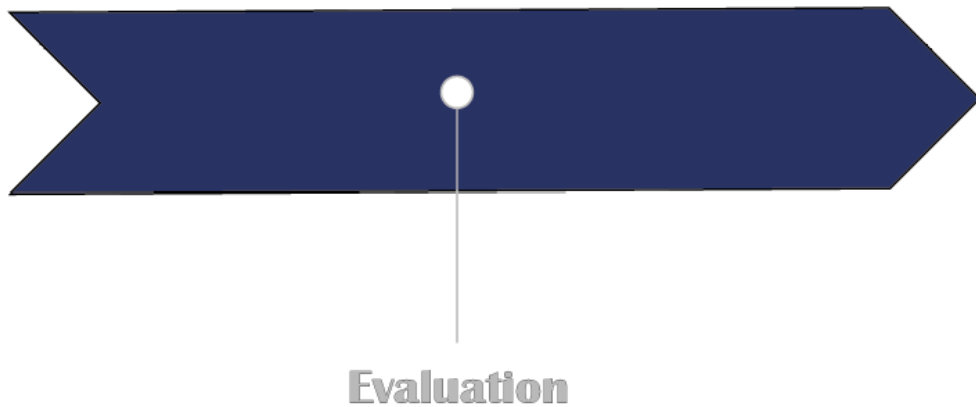
```
108 //Must be included to make sure app works with all versions of android
109 if(Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
110     NotificationChannel channel = new NotificationChannel( "Channel1", "Friend Noti", NotificationManager.IMPORTANCE_HIGH)
111     NotificationManager manager = getSystemService(NotificationManager.class);
112     manager.createNotificationChannel(channel);
113 }
114
115 // Makes first friend go home
116 NotiTest.setOnClickListener(v -> {
117     NotificationCompat.Builder builder = new NotificationCompat.Builder( context: this, channelId: "Channel1")
118         .setContentTitle(SharedData.NotiTitle)
119         .setContentText(SharedData.FriendNames.get(SharedData.ActiveGroupFriendIDs.get(1))
120             + " from " + SharedData.GroupNameArray.get(SharedData.ActiveGroup - 1)
121             + " got home safely.")
122         .setSmallIcon(R.drawable.positionicon7)
123         .setDefaults(Notification.DEFAULT_VIBRATE)
124         .setPriority(NotificationCompat.PRIORITY_HIGH);
125 }
```

Figure 5.36: Code snippet from MainMenu.Java that makes a heads-up notification

This code snippet, figure 5.36, shows how a regular notification and a heads-up notification are created. On line 110, the *NotificationManager* is set to *IMPORTANCE_HIGH*. This makes it into a heads-up notification, that is shown in Android OS's notification menu that can be opened by swiping from top of the screen to the bottom. A regular notification, without heads-up, has *IMPORTANCE_DEFAULT* instead. This type of notification needs to have an icon and a description, which a toast doesn't. On line 118, the title is loaded from an existing variable in *SharedData* and, on the next line, the description of the notification is set. The description loads the friend that got home and the group they are from and displays it. On line 122, the icon is set. With this iteration of Sapheus, we conclude the Develop phase, and move into the Deliver phase, where we evaluate Sapheus with participants from the target group, to see if Sapheus respects the design requires established in the Define phase.

6

Deliver



The fourth phase of the double diamond revolves around testing and evaluating. This is the final phase before a potential launch of the developed product, therefore it is necessary to have solved the problems met during the previous phases to ensure that testing and evaluation is relevant. The testing and evaluation done in this phase is not meant to change the design significantly but is meant to show some of the smaller undiscovered problems and to ensure that it fulfills its desired intent before launching the product to potential users. The deliver phase allows for many different methods of testing and evaluation, we decided to use questionnaires and usability testing, because both methods can cover the interactions and framework we wanted to research.

6.1 Questionnaire

A questionnaire is a set of written questions, that any number of participants can answer. Questionnaires are a quantitative method of collecting data for a survey. A questionnaire can either be open-ended or closed. The open-ended questionnaire involves giving the participant the freedom to write their own answer to a given question. In closed questionnaire, the person is not allowed to write their own answer and are instead given a set of pre-written answers as options (Sanders and Stappers, 2012). One of the limitations of the questionnaire is the lack of context, especially when used in the closed form (Gillham, 2000). With an open-ended questionnaire the participants can give some context with their written answers. An advantage with closed questionnaires is the ability to quantitatively test the data gathered with the questionnaire. One way to assemble a closed questionnaire is by using the Likert scale. When using the Likert scale, "[participants] get to choose from a predefined set of answers...[aimed at measuring] people's views on a specific topic with a greater degree of nuance" (Survey Planet, 2022). To measure people's view, the Likert scale "usually contains 5 or 7 response options , ranging from strongly agree to strongly disagree, with differing nuances between these and a mandatory mid-point of neither agree nor disagree" (Survey Planet, 2022). An overview of the two different Likert scales can be seen in figures 6.1 and 6.2.



Figure 6.1: 5-point Likert scale



Figure 6.2: 7-point Likert scale

We chose to make a closed questionnaire using a 7-point Likert scale as an interval scale. Viewing the responses derived from the Likert scale as interval, presents a limitation. The main limitation encountered when working with the Likert Scale is that, although it is considered an interval scale, we cannot ensure that the participant agrees what the midpoint between neutral and strongly agree is. Research has been conducted and "shows that people perceive disagree to be close to strongly disagree and agree to be close to strongly agree, making the Likert scale an ordinal scale; however, when an adjective such as "slightly" is added to disagree and agree, people perceive disagree

slightly to be close to the halfway point between strongly disagree and neutral, and agree slightly to be close to the halfway point between neutral and strongly agree” (Chyung, 2019), as depicted in figure 6.3 below.



Figure 6.3: Perceived 7-point Likert scale

With this in mind, we constructed a 7-point Likert scale questionnaire, to see how much participants representing our target group would agree or disagree with certain statements regarding safety and connectedness. To ensure that we were testing the app itself, and not how the participants feel about the statements in general, we had two groups of participants; one group that responded with the Sapheus app and another that responded without the app. This was done to see if the presence of the app altered the participant’s perception of safety and connectedness. Our questionnaire consisted of 12 questions, 5 regarding connectedness, 5 regarding safety, and 2 regarding formalities. To make the questions regarding connectedness we got inspiration from these two pre-established questionnaires (Lee and Lohmeier, 2011) (Ijsselstein et al., 2009). The 5 questions about connectedness are listed below, as well as the reasoning for using this list of questions to assess connectedness.

1. I want to order something for my friends and I, I know I can contact them.
2. I worry about my friends well-being.
3. I feel that friends care about me.
4. I am likely to meet up with friends throughout the night.
5. I am able to text my friends and get a response.

These questions were chosen to assess if, and if so, how much the participant was connected to the people around them, both physically and emotionally. These questions showed us how people connect, both physically but also online, which was of importance to this project, since one of the main themes we worked around was connecting people. The following 5 questions regard the participant’s perception of safety, a central focus of our app. When creating these questions, we were inspired by a pre-established questionnaire regarding public safety in Seattle (Helfgott and Parkin, 2017).

1. I have gotten lost from my friends on a night out, I am sure I can find them again.
2. I am sure my friends are safe
3. I am able to get help from a friend.
4. Friends know my whereabouts.
5. I feel safe when I’m out with my friends.

These questions were included to specifically assess how the presence of Sapheus altered the participant's perception of their own safety, as well as the safety of their friends. This further illustrates the importance of this questionnaire having two groups of responders. By distinguishing between participants who replied with Sapheus and without, we are able to see how the app affects perceived safety and connectedness, further illustrating if Sapheus fulfills the intended purpose of use. To ensure that the order of the statements have no effect on the responses, the order of the questions was randomized by Google Forms. With the questionnaire established, it was sent to people within the target group and the following section details the collected data and how we used this data to gather insight used to evaluate Sapheus.

6.2 Questionnaire Results

With the data collected from the questionnaire we wanted to test if the presence of Sapheus made a difference regarding the participants feelings of safety and connectedness. These were evaluated with t-tests and a Cronbach's alpha test, to evaluate the internal consistency relative to the themes of the questions. As a rule of thumb when making tests as the t-test "[usually] you want at least 15-20 participants: smaller studies may miss potentially interesting results. The inclusion of more participants gives you more statistical power" (Feng et al., 2017, p. 459). We were hoping to get about 30 responses to the questionnaire equally split, so 50% of the participants answered with Sapheus and 50% without. With this we would have a reasonable number of participants to test the results. In actuality, the questionnaire had 20 responses, 60 percent responded with Sapheus and 40 percent responded without. We recognize that our sample size isn't large enough to make the following t-test but because we wanted to test if there was a significant difference between the two groups because of the presence of Sapheus regarding connectedness and security, the t-tests were made anyway with that flaw in mind. To make the t-tests, all the data from the questionnaire was converted into a numbered point system, which followed our 7-point Likert scale, with 1 being the worst and 7 being the best as seen in figure 6.4.

Connecte	Q1	Q2	Q3	Q4	Q5	Total	Safety	Q6	Q7	Q8	Q9	Q10	Total
WITH the	2	3	7	7	3	22	WITH the	4	6	6	5	5	26
WITH the	2	4	4	7	3	20	WITH the	7	6	5	5	7	30
WITH the	6	7	6	6	5	30	WITH the	5	5	5	6	6	27
WITH the	6	6	6	6	5	29	WITH the	3	6	6	5	7	27
WITH the	6	6	6	7	4	29	WITH the	7	6	6	6	6	31
WITH the	6	5	6	7	5	29	WITH the	5	5	6	5	6	27
WITH the	5	7	7	7	7	33	WITH the	5	6	5	7	6	29
WITH the	5	1	6	6	5	23	WITH the	5	4	5	6	7	27
WITH the	6	6	7	7	5	31	WITH the	7	6	6	7	7	33
WITH the	5	6	6	5	5	27	WITH the	6	6	5	7	7	31
WITH the	6	7	7	2	5	27	WITH the	4	6	5	6	6	27
WITH the	6	4	4	5	6	25	WITH the	6	5	6	6	6	29
WITHOUT	5	5	6	6	5	27	WITHOUT	6	5	6	5	7	29
WITHOUT	5	6	2	5	5	23	WITHOUT	5	5	4	3	4	21
WITHOUT	7	7	7	4	7	32	WITHOUT	3	5	7	7	5	27
WITHOUT	7	7	5	4	2	25	WITHOUT	6	4	6	4	7	27
WITHOUT	6	6	7	5	5	29	WITHOUT	7	6	7	6	7	33
WITHOUT	6	1	7	7	6	27	WITHOUT	6	6	7	7	7	33
WITHOUT	4	6	6	4	5	25	WITHOUT	6	5	6	5	6	28
WITHOUT	6	4	6	6	6	28	WITHOUT	7	5	6	4	6	28
WITHOUT	7	6	7	1	7	28	WITHOUT	7	7	7	7	7	35
variance	1,768707	3,133787	1,569161	2,721088	1,569161		variance	1,578231	0,535147	0,630385	1,269841	0,680272	

Figure 6.4: Questionnaire responses table with point system

The hypothesis was that the presence of Sapheus makes a positive difference in the participants perception of their connectedness and safety. An unpaired t-test was performed in two parts by separating the data into the two themes, safety and connectedness. The null-hypothesis for both themes was that the presence of Sapheus does not make a difference on the participants perception of connectedness or safety. The t-test regarding connectedness resulted in a t-score of 0.0 and p-value of 1.0, this means that the two groups have equal results regarding connectedness, as seen in figure 6.5.

Connectedness		
The two-tailed P value equals 1.0000		
t = 0.0000		
df = 8		
standard error of difference = 0.333		
Group	WITH	WITHOUT
Mean	5.420	5.420
SD	0.550	0.502
SEM	0.246	0.224
N	5	5

Figure 6.5: Results of Connectedness t-test

When running the t-test for the safety related questions, the result was a t-score of 0.22 and a p-value of 0.82, as seen in figure 6.6. In both cases the p-value is greater than 0,05 which means that the null-hypothesis was accepted, meaning that the presence of Sapheus didn't affect perceived safety for our test sample.

Safety		
The two-tailed P value equals 0,8284		
t = 0,2239		
df = 8		
standard error of difference = 0,268		
Group	WITH	WITHOUT
Mean	5,720	5,780
SD	0,390	0,455
SEM	0,174	0,203
N	5	5

Figure 6.6: Results of Safety t-test

From this test, it is evident that the presence of Sapheus did not make a difference on the participants perception of their connectedness or safety. With the t-tests done, we wanted to know how well the questionnaire measured safety and connectedness. This was done by conducting a Cronbach's Alpha test, where the two groups were combined, but separated by theme, the results of the test can be seen in figures 6.7 and 6.8.

Connectedness	
True data	
Questions	5
sum of variance	10,7619
variance of total	10,56236
cronbach's a	-0,02362
Reversing 1s	
Questions	5
Sum of Variance	8,12
Variance of total	13,09
Cronbach's a	0,47

Figure 6.7: Results of Connectedness Cronbach's Alpha test

Safety	
Questions	5
Sum of variance	4,693878
Variance of total	9,297052
Cronbach's a	0,618902

Figure 6.8: Results of Safety Cronbach's Alpha test

When interpreting the results of the Cronbach's Alpha test, as a rule of thumb, the resulting value should be greater than 0,9 for the test to be considered good. Our results produce a lower alpha value, meaning the scale reliability of the data and internal consistency is worse. We would have accepted a value of 0,7 and above to consider our questionnaire questions good. When looking at our result from connectedness, the initial alpha value is negative which indicates that there is a negative average covariance, which makes the test invalid. This is likely caused by having a very high sum of variance because some questions had primarily strongly agree answers but one or two people have answered strongly disagree, which results in a high variance but it could also be caused by having very few items or questions which tends to cause a lower reliability (Tavakol,

2011). A workaround for this could be to omit entire items or questions. In our case, this is not a possible solution as we only have five questions regarding this theme. If we omitted the questions with a high sum of variance, we would be left with only two questions and an even lower reliability. We suspect that the three strongly disagree answers might be because the questions were misunderstood, as they were outliers and, in each case, were opposite from the participants answers to the other questions. Even though we know that the following test is not valid, we decided to conduct a new Cronbach's Alpha test where the three answers was changed to strongly agree. This resulted in an alpha value of 0.47 which could still be considered questionable and is considerably lower than our goal alpha value of 0,7. The results of the Cronbach's Alpha test regarding the five safety questions were better than for the questions regarding connectedness. With an alpha value of 0,61, the internal consistency of the questions regarding safety is still lower than our goal, but closer to our acceptance goal of 0.7. In general, the results from our questionnaire have a questionable internal consistency. It also shows that the answers from the questionnaire are quite varied which makes it hard to make conclusions regarding the two themes (Tavakol, 2011). As detailed in Future Works, we would like to make another questionnaire based on the information collected from the Cronbach's alpha test, to improve the questions and attain a better understanding of how our target group views Sapheus. After having conducted t-tests and Cronbach's Alpha tests from the data collected from the questionnaire, we wanted to conduct usability tests to get a better understanding of the problems regarding the use of Sapheus. The usability tests in detailed in the following section.

6.3 Usability Testing

Usability testing as a research method is capable of uncovering problems within the use of an interface or software. A common method used during a usability test revolves around making the participant think aloud by narrating their thoughts and actions. The thinking aloud method allows the researchers to hear the participants point of view and therefore get a glimpse of which aspects of the interface works and which causes usability problems. By using the think aloud method to conduct a usability test, improvements can be made to the system by listening to their feedback and following their behaviour through the system to make improvements. Usability testing is conducted with the participant and a facilitator in the same room. The facilitator is meant to be the translator between the participant and the research team behind the usability test, included to give the participant the tasks included in the test, join the participant on their process through solving the task, and to remind the participant to think aloud, if necessary. (Nielsen, 1993, p. 165-206).

We decided to conduct usability tests with people in the target group previously defined. To conduct our usability test, we invited four participants to partake, and they consented to the usage of the data collected during their individual test. We specifically applied the think aloud method, as detailed above. Along with the participant, a facilitator was also

present in the room to assist the process of the usability tests. The usability test was conducted on a phone with Sapheus and recorded, so we could return to the recordings to specify problem areas within the app. The tests were conducted by having the participant go through five different task scenarios, with a full description of their intent for completing each task as listed below, facilitated through the think aloud method. The scenarios given to each participant were:

1. **Activate group:** You meet up with a group of your friends for night in Jomfru Ane Gade. You're about to head out the door and your friend, Chris, asks if you could activate your group from last week, the StreEet group specifically. Activate this group.
2. **Create meeting point:** Before heading out, you all want to ensure that you're together at 22.00 to have a beer to celebrate Chris's birthday. Create a meeting point at 22.00.
3. **Check if a group member has gotten home:** It's later in the evening and a couple from StreEet has left. Check your group member's status. Who has gotten home? Who is still partying?
4. **Request help:** You're lost in down-town Aalborg and need help from someone you're partying with to get back to Jomfru-Ane Gade. Send a request for help.
5. **I'm Safe:** You've safely gotten home and to bed. Tell your group that you've gotten home.

These tasks were chosen for the usability test because, collectively, they covered all essential functionalities of our app, as indicated by the bold text above, illustrating if we had made our functionalities understandable and easy to use, specifically for novice users. With the first and second scenarios, we aimed at testing the connection between friends, by establishing a group and creating a meeting point, which was specifically aimed at improving the connectedness when partying, because the friends in a group could meet up and find one another, if they had separated. The last three were included in the usability test to assess the safety net that the usage of the app establishes between friends. With the tests conducted, the results from the usability tests are detailed in the following section.

6.4 Usability Test Results

The evaluation of our usability tests was done by watching the recording of each test and noting the problems each participant encountered. This was done to gain an understanding of how difficult it is to navigate and use the app, resulting in finding areas that need improvement. The following three usability problems were uncovered by the usability tests:

1. Assumes that *Activity* can be used to make a meeting point
2. Uses different touches on map to create meeting point
3. Single touch to activate *I'm Safe*

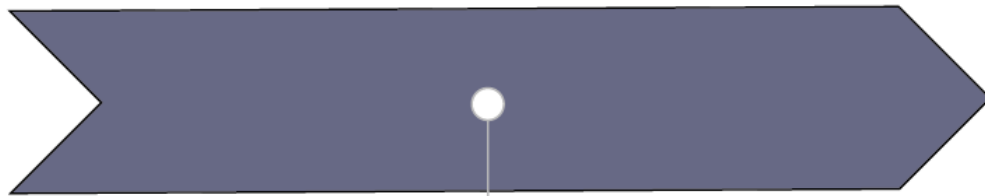
The first found usability problem occurs when the participant is asked to create a meeting point. On the group page, the chat feature, called *Activity*, mislead all the participants, resulting the assumption that the meeting point could be created here. This problem is result of a lack of "preventing errors" (Nielsen, 1993, p. 145) and "good error messages"(Nielsen, 1993, p. 142). All participants mentioned that they thought a meeting could be created here because the activity chat showed a meeting point that had already been created. This was a design error because the chat was a simulated, static chat for the purpose of the test, rather than a dynamic chat that updated with activity in the app. This problem is a major usability problem that needs to be addressed alongside Future Works, as detailed in chapter 8.

The second problem occurred when the participants had navigated to the map. Here, they try to single or double tap on the map to create a meeting point, which doesn't work. Only one participant figures out that a long press is needed to create a meeting point. The other three participants did not complete the task. With 3/4 participants not being able to complete this task, the problem of creating a meeting point needs to be corrected. The main reason for this problem is due to a lack of "feedback" (Nielsen, 1993, p. 134). This problem is a major usability problem, that needs to be addressed alongside the chat problem mentioned above.

The last usability problem occurs when the participants single touched the *I'm Safe* button to activate it. When single touching on this button, a toast notification pops up and says that a long press is required. After reading this, all the participants successfully completed the task. We defined this as a cosmetic problem, because, with the application of Nielsen's "feedback" (Nielsen, 1993, p. 134) heuristic, it was solved. But we wanted to include it to illustrate the effect of including proper feedback. Two participants noted that they would have liked the toast notification when trying to create a meeting point. With these problems found, the main usability concern of Sapheus is a lack of feedback. Some ideas for how to solve this will be discussed in chapter 8, Future Works.

7

Discussion



Alternatives

Through this project we have decisions that have had a great effect on the process and the outcome. Some of the choices and their effect will be discussed in this chapter.

7.1 Exploring and Defining the Scope

Before deciding on a concept for this project, we used a great amount of time exploring the scope to get a shared foundation of knowledge that we could build our design concepts upon. We did this by researching what it means to connect people and on what basis people connect. The amount of time used on this part of the process was well spent as it gave us a shared understanding from the beginning of the process. We later in the process decided between creating a party focused or a safety focused app. It would have been an advantage for us if we had made this decision while exploring the scope of the project as it would have given us a more focused Define phase. In hindsight, this would have been an advantage, but we still got to a great result that was based on the data we collected from the questionnaire and interviews in the Discover process.

7.2 Nightlife Setting

The foundation of this product has been a nightlife setting, which made a lot of sense as the tragedies surrounding the disappearances of Mia Stevn and Oliver Lund unfolded at the time we were introduced to the theme of this project and we were all affected by this. We realize that the product we developed could be used in different settings than nightlife as it could also be used by groups during the daytime to keep each other safe. If Sapheus was made with any other setting in mind, some aspects of the app would have to be expanded upon and adapted. One of the aspects that could be adapted is the *Help* page, where different types of help buttons could be added to make it fit another settings. Choosing a setting in the beginning of this project gave us a strong foundation to build our concept upon which resulted in a product that is very focused on a nightlife setting.

7.3 Android Studio

When deciding what to program our app in, we chose to use Android Studio. This decision made a lot of sense at the time as most of the group members had little to no experience programming an app and there is a lot of great resources and documentation for using Android Studio. After using Android Studio and gaining experience in coding, we see the validity in moving our app to Xamarin because this would make it possible for different operating systems to use the app, expanding the audience of Sapheus. This will be further discussed in chapter 8, Future Works.

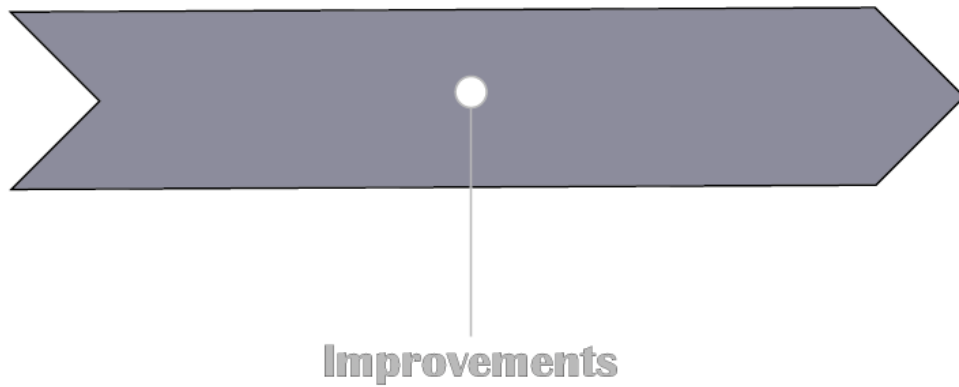
7.4 Evaluation of Sapheus

To evaluate Sapheus, we created a questionnaire and conducted usability tests. For the questionnaire, the participants were divided in two groups. We asked our fellow IxD4 students to answer the questionnaire with the app, since we were all present in the lab. Since we tested the app in a lab-setting, the usage of Sapheus seemed artificial compared to the ideal usage. The ideal setting for answering the questionnaire with the app would be on the weekend after the participants have been to a party, but this would create some

ethical problems as we would have to encourage the participants to get drunk. We also assume that it would limit our number of participants. For these reasons, we think that the decision of having the participants answer the questionnaire in the lab was the best decision for this project. When creating the questionnaire, we wanted to use preestablished research questions, but finding questions that fit our themes turned out to be a challenge. Instead, we took inspiration from questionnaires that measured safety and connectedness in different settings. We realize that making the questions ourselves was not an ideal decision, but to fit the themes, taking inspiration was a great decision. To test if the questions we used had internal consistency or not, we made a Cronbach's Alpha test. This showed that the questions did not measure the themes very well, which might also have influenced the result of the t-tests. Possible solutions for this problem will be detailed in chapter 8, Future Works. To summarise our process, we feel we have made some valid decisions with the knowledge we had at the time, but if we had to start over, with the knowledge we have gained, we expect the outcome of the process to look different.

8

Future Works



In this chapter, potential future works is described based on the evaluations conducted in chapter 6, Deliver. Additionally, future works that could be conducted to further improve the general workings of Sapheus are also outlined.

Based on our usability test, one overall problem emerged. The problem was based in a lack of feedback, specifically tied to the creation of a meeting point. As mentioned in chapter 6, Deliver, all the usability test participants struggled to create a meeting point, which illustrates a design error. As mentioned in chapter 6, the participants requested an error message to suggest how a meeting point is created, further proving that our design is lacking "good error messages" and "preventing errors" (Nielsen, 1993, p. 142-145). To correct this design error, a type of feedback or tutorial in creating a meeting point needs to be coded into the app, so that users can get help and error messages, to aid in the usability of the app. This directly ties into Nielsen's heuristics of "Feedback" (Nielsen, 1993, p. 134) and "Help and Documentation" (Nielsen, 1993, p. 148). With regards to the inclusion of a tutorial, this could also be applied to the *I'm Safe* feature, since multiple participants encountered the toast notification regarding how a long touch is required to engage *I'm Safe*. Since this problem was categorized as cosmetic, it can be fixed, but it doesn't take priority should this work be completed, therefore, it is tied into fixing a larger problem. The last problem encountered in the usability test was the chat. To fix this problem, a chat functionality needs to be coded and added to the *Groups* page. With all the problems from the usability test addressed, the overall additional development needed on Sapheus is detailed.

Once these usability problems are fixed, using the data collected when analysing the questionnaire from chapter 6, Deliver, another questionnaire would need to be created. Based on the Cronbach's Alpha test conducted based on the questionnaire, the questions included in a new questionnaire would need to be altered to fit within the findings of this test, to ensure that each new question tests the intended topic and additional questions should be made. This must be tested multiple times or until getting an acceptable alpha value. Additionally, this questionnaire would be given to people who had recently been to a party, to see if Sapheus would be used within the intended setting, since we can only speculate at the time of this project's conclusion. This would show if Sapheus fulfills a purpose for people while partying, or if it is viewed as an annoyance or hindrance.

To make Sapheus an app that facilitates users in connecting with one another and creates a network between users, a server needs to be established. The server is an essential feature needed to make the app functional between users. The server feature might affect the current code and some of it needs to be rewritten to properly interact with the server. Additionally, a database needs to be added, used to store necessary user information. Since we need to store certain information regarding Page 70 Chapter 8. Future works AAU each user, so that the app can function as intended, extensive research regarding the Danish *Persondataloven* (translation: the personal data law) and the Danish *Databeskyttelsesloven* (translation: the data protection law). *Persondataloven* is the general law dictating how personal information of Danish citizens should be stored and handled (Justitsministeriet, 2000). *Databeskyttelsesloven* is a directive within

Persondataloven, which further specifics how this data is stored and how it may be shared (Justitsministeriet, 2018). Prior to launching Sapheus, extensive research within these two laws must be conducted, to ensure that the database and server are coded in accordance to Danish law. Lastly, to further develop Sapheus, it should be coded in an environment that supports both Android and IOS operating systems. We chose to code in Android Studio based on our minimal experience with coding an interface. This was an excellent platform for us, to learn how to code an app, but, if we had more experience in coding, we would have liked to code in an environment that is compatible with both operating systems, for example Xamarin. This would have allowed Sapheus to be available to all smartphone users, rather than limiting the potential users to their choice of phone. To further develop Sapheus, it would be ideal to expand the target group, not only allowing Android users to have access to Sapheus. Due to our minimal coding experience, we chose to code in Android Studio, so that we could learn the basics of coding an app, but we would like to expand the reach of Sapheus by coding the Xamarin, when further developing the concept.

9

Conclusion



This chapter concludes our process and summarizes our results.

By developing Sapheus, we proposed an interactive solution that increases perceived safety by connecting friends. In our process of following the Double Diamond of Design, we were able to explore design proposals that aided in solving problems illustrated by users involved. A main concern of the involved users was the safety of their friends, an essential element implemented in Sapheus. To develop Sapheus, we learned how to code in Android Studio, which provided us with great insight that can be brought into other coding-based projects. To evaluate Sapheus, we conducted t-tests and Cronbach's alpha test based on results from a questionnaire, that illustrated certain problems related to the sample test size, which needs to be repeated. Additionally, we conducted usability tests, that emphasized certain usability problems that must be repaired prior to conducting further evaluations. It has been thoroughly informative to work with a coding-based app that centered around a current case, giving our app real-life applications.

Bibliography

- Baeldung. A Guide to the Static Keyword in Java, 2021. URL <https://www.baeldung.com/java-static>. Accessed 05/07/2022.
- British Design Council. A study of the design process, 2022b. URL [https://www.designcouncil.org.uk/sites/default/files/asset/document/ElevenLessons_Design_Council%20\(2\).pdf](https://www.designcouncil.org.uk/sites/default/files/asset/document/ElevenLessons_Design_Council%20(2).pdf). Accessed 05/18/2022.
- British Design Council. What is the framework for innovation? Design Council's evolved Double Diamond, 2022a. URL <https://www.designcouncil.org.uk/news-opinion/what-framework-innovation-design-councils-evolved-double-diamond>. Accessed 05/18/2022.
- Butler, J., Holden, K. and Lidwell, W. Universal Principles of Design. Rockport Publishers, 2010.
- Christensen, S. K. Efter intens eftersøgning: Politiet har fastlagt 22-årige Mias færden, 2022a. URL <https://migogaalborg.dk/efter-intens-eftersoegning-politiethar-fastlagt-mias-faerdig/>. Accessed 05/09/2022.
- Christensen, S. K. Endelig bekræftelse: Mia Skadhauge Stevn fundet død, 2022b. URL <https://migogaalborg.dk/endelig-bekraeftelse-mia-skadhauge-stevn-fundet-doed/>. Accessed 05/09/2022.
- Chyung, Y. Evidence-Based Survey Design: Do You Agree or Somewhat Agree?, 2019. URL <https://www.td.org/insights/evidence-based-survey-design-do-you-agree-or-somewhat-agree>. Accessed 05/18/2022.
- Clante, C., Eriksen, C. T., and Trøiborg, I. Mia Skadhauge Stevn er bekræftet død: Det ved vi om sagen, 2022. URL <https://www.dr.dk/nyheder/indland/maend-sigtet-drab-efter-ungkvindes-forsvinden-det-ved-vi-om-sagen/>. Accessed 05/18/2022.
- Digitaliseringsstyrelsen, S. S. Institut, S. for Patientsikkerhed, Sundhedsministeriet, and Sundhedsstyrelsen. Trin-For-Trin-Guide, 2020. URL <https://smittestop.dk/uploads/trin-for-trin-guide.pdf>.
- Feldmar, R. The rise of the smartwatch: Will smartphones become redundant?, 2020. URL <https://www.resco.net/blog/smartwatch-will-smartphones-become-redundant/>.
- J. H. Feng, J. H., Hochheiser, H. and Lazar, J. Research Methods in Human-Computer Interaction. Morgan Kaufmann Publishers, 2017.
- Gillham, B. Developing a Questionnaire. Continuum International Publishing Group, 2000.
- Harris, M. J. Color Theory and its Applications in Marketing as it Relates to Color Psychology, 2018. URL https://www.researchgate.net/publication/329487553_Color_Theory_and_its_Applications_in_Marketing_as_it_Relates_to_Color_Psychology.

- Health, C. P. Content Analysis, 2022. URL <https://www.publichealth.columbia.edu/research/population-health-methods/content-analysis#:~:text=Content%20analysis%20is%20a%20research,words%2C%20themes%2C%20or%20concepts>. Accessed 05/18/2022.
- Helfgott, J. B. and Parkin, W.. 2017 Seattle Public Safety Survey Results, 2017. URL https://www.seattle.gov/Documents/Departments/Police/Reports/2017_CITYWIDE_Seattle_Public_Safety_Survey_Report.pdf?fbclid=IwAR2He37hlzpqTJovtyR4ko8zG-PQrfJv9eK1-huF9Pf3zSfLau9MG6SzkRc.
- Ijsselsteijn W. A., Kort, Y. D. and Smolders, K. C. H. J. Social connectedness: Concept and measurement, 2009. URL https://www.researchgate.net/publication/220992602_Social_connectedness_Concept_and_measurement.
- Julsgaard, L. E. 22-ÅRIGE SUNE KNOGLER I DØGNDRIFT - FREDAG ÅBNER HAN SAFE HOUSE I GADEN, 2022. URL <https://aalborgnu.dk/nyheder/22-aarige-sune-knogler-i-dogndrift-fredag-aabner-han-safe-house-i-gaden/93858ddb-823f-4d9d-b431-6a2c676f25ee?fbclid=IwAR0lqD6qKNnTibJQaqtODzUuLnH4EZmErlxSHVvZkV4gMsgl6enwv8dg9Q>. Accessed 05/09/2022.
- Jungersen, R. Efter to tragiske sager diskuterer Aalborg, hvordan nattelivet kan gøres mere sikkert, 2022. URL <https://www.dr.dk/nyheder/regionale/nordjylland/efter-tragiske-sager-diskuterer-aalborg-hvordan-nattelivet-kan-goeres>. Accessed 05/09/2022.
- Justitsministeriet. Persondataloven, 2000. URL <https://www.retsinformation.dk/eli/lta/2000/429>.
- Justitsministeriet. Databeskyttelsesloven, 2018. URL <https://www.retsinformation.dk/eli/lta/2018/502>.
- Jørgensen, E. K. Nordjyllands Politi: Har fundet Oliver i Aalborg Havn, 2022. URL <https://migogaalborg.dk/mand-fundet-dog-i-aalborg-havn/>. Accessed 05/09/2022.
- Lee, S. and Lohmeier, J. H. A school connectedness scale for use with adolescents, 2011. URL https://www.researchgate.net/publication/239787465_A_school_connectedness_scale_for_use_with_adolescents.
- MasterClass Staff. A Guide to the 5 Levels of Maslow's Hierarchy of Needs, 2020. URL <https://www.masterclass.com/articles/a-guide-to-the-5-levels-of-maslows-hierarchy-of-needs#what-are-the-5-levels-of-maslows-hierarchy-of-needs>. Accessed 05/18/2022.
- Møller, E. Det blev dyre dråber til unge værtshus-gæster, 2016. URL <https://www.tvsyd.dk/tonder/det-blev-dyre-draber-til-unge-vaertshus-gaester>.
- Nielsen, J. Usability Engineering. Academic Press, 1993.
- Nordjyllands Politi. To anholdte – politiet arbejder på to adresser, 2022. URL <https://politi.dk/no>

rdjyllands-politi/efterlyste-og-savnede-personer/090222pmsag/2022/02/09. Accessed 05/09/2022.

Sanders, E. B.N. and Stappers, P. J. Convivial toolbox: generative research for the front end of design. BIS Publishers, 2012.

Shrestha, P. Convergent Vs Divergent Thinking, 2017. URL
<https://www.psychestudy.com/cognitive/thinking/convergent-vs-divergent>. Accessed 05/18/2022.

Strømgaard, H. Politiet: - Vi er til stede i bybilledet for at skabe tryghed, 2022. URL
<https://nordjyske.dk/nyheder/aalborg/politiet-vi-er-til-stede-i-bybilledet-for-atskabe-tryghed/5ffcd9b7-fda8-42a1-ab66-6aaf1257b29f>. Accessed 05/09/2022.

Supercolor. The Meaning of the Color Blue, 2020. URL
<https://www.supercolor.com/blog/the-meaning-of-the-color-blue/>. Accessed 05/17/2022.

Survey Planet. Likert Scale: How to Interpret the Results of a Satisfaction Survey Scale?, 2022. URL
<https://blog.surveyplanet.com/likert-scale-how-to-interpret-the-results-of-a-satisfaction-survey-scale>. Accessed 05/18/2022.

Tavakol, M. Making sense of Cronbach's alpha, 2011. URL
<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4205511/>. Accessed 05/21/2022.

Wharton, C. The Psychology of Design: The Color Green, 2019. URL
<https://www.impactplus.com/blog/the-psychology-of-design-the-color-green>.